Declarative distributed algorithms as axiomatic theories in three-valued modal logic over semitopologies

Murdoch J. Gabbay

gabbay.org.uk

Contents

Abstract

We illustrate how to formally specify distributed algorithms as declarative axiomatic theories in a modal logic, using as illustrative examples a simple voting protocol, a simple broadcast protocol (Bracha Broadcast), and a simple agreement protocol (Crusader Agreement). The methods scale well and have been used to find errors in a proposed industrial protocol. The key novelty is to use modal logic to capture a declarative, high-level representation of essential system properties – the logical essence of the algorithm – while abstracting away from explicit state transitions of an abstract machine that implements it. It is like the difference between specifying code in a functional or logic programming language, versus specifying code in an imperative one. Thus we present axiomatisations of *Declarative Bracha Broadcast* and *Declarative Crusader Agreement*. A logical axiomatisation in the style we propose provides a precise, compact, human-readable specification that abstractly captures essential system properties, while eliding low-level implementation details; it is more precise than a natural language description, yet more abstract than source code or a logical specification thereof. This creates new opportunities for reasoning about correctness, resilience, and failure, and could serve as a foundation for human- and machine verification efforts, design improvements, and even alternative protocol implementations.
The proofs in this paper have been formalised in Lean 4.

Keywords: Distributed algorithms, three-valued modal logic, semitopology, Declarative Bracha Broadcast, Declarative Crusader Agreement
This author's version of the document was compiled on 11 May 2026.

1 Introduction

1.1 Introducing the declarative, axiomatic approach

We offer a new approach to analysing distributed algorithms, by exhibiting them as axiomatic theories in a purpose-built three-valued modal logic (one with a 'middle' truth-value **b** between **t** and **f**) over a semitopology (like topology but without the condition that the intersection of two open sets is necessarily open) [12,13]. This captures a declarative essence of the essential system properties.

Nonempty open sets of a semitopology represent quorums. If a predicate returns **t** (true) or **f** (false) this reflects correct behaviour, and if it returns **b** (both / byzantine) this reflects faulty behaviour.

The effect is similar to the difference between an explicit for-loop to apply a function `f` to each element of a list `l`, and just writing `map f l`: we abstract declaratively to the essential logic of the algorithm.

This paper is designed to introduce the basic ideas of this new declarative / axiomatic approach. These techniques can be applied to larger examples, including modern industrial-scale ones, but here we consider simpler algorithms so that we get a clearer view of the basic idea. We consider Bracha Broadcast and Crusader Agreement: these are short but non-trivial examples of the genre; they illustrate our techniques; and this provides an accessible introduction to and exposition on this new way to apply logic to study distributed algorithms.

In this paper, our examples (voting, broadcast, agreement) will be toy and/or textbook algorithms. These toy examples will not, and are not intended to, showcase the full power of our proposed approach relative to standard distributed computing methods. This is a feature, not a bug: we show how to attain simplicity via abstraction in logic, using familiar examples as case studies. Attaining this simplicity is mathematically non-trivial, and we shall see that there is plenty of subtlety to discuss along the way (and much mathematical beauty, too).

The proofs in this paper have been formalised in Lean 4 and sources are freely available for download [28].

As for scaling up, the techniques illustrated in this paper scale well. In fact, more complex algorithms create *more* scope to benefit from the abstractions in this paper by eliding implementational detail:

(1) In other work we apply these ideas to Paxos [17]. [1]
(2) Since this paper was conceived, we scaled up declarative techniques to analyse *and find errors in* a proposed industrial protocol called *Heterogeneous Paxos* [32,33]. The complexity of Heterogeneous Paxos had made it prohibitively hard to debug using traditional, imperative techniques. We completed a declarative analysis, found the error, and we used declarative techniques to design a replacement protocol [16], whose correctness was formally verified in Lean 4 [18] by Hart. [2]

A note on terminology: we will use 'distributed algorithm' and 'distributed protocol' synonymously. The difference is in emphasis: if we write 'algorithm' we imply an omniscient view of the system from the outside, whereas if we write 'protocol' we imply a view of the system from a participant communicating with other participants according to the rules of the algorithm. However, this difference in emphasis just reflects a switch in point of view. The reader can safely identify both words with the same meaning throughout.

1.2 Motivation: distributed algorithms

Distributed algorithms are algorithms that run across multiple participants, such that the overall behaviour of the system emerges from their cooperation. Contrast with a *parallel* algorithm, which also runs across multiple participants but which would still make sense running (albeit more slowly) on a single participant. [3] Distribution can provide benefits in resilience and scalability which could not be obtained any other way.

A fundamental challenge of distribution is that we cannot assume that all participants in the algorithm are correct: we need to be resilient against failure, and against hostile (also called *byzantine*) behaviour. Distributed algorithms are notoriously difficult to design, specify, reason about, and validate, because of the inherent difficulty of coordinating multiple participants who may be faulty, hostile, drifting in and out of network connectivity [26,29], and have different local clocks, states, agendas, etc.

This gives the field of distributed algorithms a very particular flavour. Everything revolves around this central question: how does our algorithm guarantee prompt and reliable results for correct participants, when parts of the system may be faulty?

Blockchain consensus algorithms are a good source of examples, both for distributed algorithms and for how things can go wrong. [4] Solana (`https://solana.com`) has experienced instances where its consensus mechanism contributed to temporary network halts [36,34]; Ripple's (`https://ripple.com`) XRP consensus protocol has been analysed for conditions in which safety and liveness may not be guaranteed [4]; and Avalanche's (`https://www.avax.network`) consensus protocol has faced scrutiny regarding its assumptions and performance in adversarial scenarios [5]. Even mature blockchains like Ethereum are prone to this: e.g. *balancing attacks* [25] and *ex-ante reorganisation* (*reorg*) attacks [30] exploit vulnerabilities in Ethereum's current protocol. Even as this paper was prepared, XRP consensus experienced a slowdown and stoppage, which was addressed in a GitHub commit. [5]

Thus, subtle flaws and/or ambiguities in protocol design have significant consequences, emphasising the need for rigorous analysis and well-defined specifications in this complex and safety-critical field.

Meanwhile in the real world, such analysis and specification may be abbreviated or skipped altogether:

---

[1] The logic and models needed to do this are different, reflecting that Paxos is a different algorithm, but the basic 'declarative' principle is similar.

[2] Hart also worked on the original pseudocode/English specification. He reported that he found the axiomatic/declarative specification and proofs much easier to work with.

[3] A GPU rendering algorithm is a good example of a typical parallel algorithm. It could run on a single-threaded CPU, and that would make perfectly good sense; it would just be too slow. In contrast, a blockchain is a good example of an algorithm that is fundamentally distributed and not parallel: it could still run on just one single participant, but that would completely defeat the point of the blockchain, which is precisely to *be* distributed across many participants.

[4] This work was motivated by analysis of blockchain systems.

[5] cryptodamus.io/en/articles/news/xrpl-meltdown-64-minute-outage-xrp-ledger-s-february-4th-2025-halt-decoded (permalink) and github.com/XRPLF/rippled/pull/5277 (permalink).

that is, in industrial practice a protocol may go from an initial idea written in English or pseudocode to an implementation, without a rigorous analysis. This is because such analysis would be just too expensive. Part of the motivation for this work is, via logic, to provide a precise, compact, nontrivial, yet (relatively) lightweight methodology for specifying and reasoning about distributed protocols. Returning to the example above of an explicit for-loop versus a map function, the idea is to attain compact proofs by being as declarative and static as possible, while preserving the essence of the algorithm.

1.3 Map of the paper

- We give a simple voting algorithm in Section 2. This gives us an opportunity to introduce semitopologies and three-valued logic.
- We define the syntax of a modal logic in Section 3: this is the formal language which we will use for our axioms and correctness properties.
- We study Bracha Broadcast in Section 4.
- We study Crusader Agreement in Section 5.
- Finally, we conclude with Section 6.

2 Motivation and an illustrative example: voting

We will warm up by studying a voting algorithm. This is simpler than Bracha Broadcast but is still enough to invoke much of our machinery, including the three-valued logic and semitopologies.[6] The axiomatisation is in Figure 3. In Example 2.1.1 we describe the protocol informally:

2.1 Protocol description

Example 2.1.1 Fix $f \geq 1$ and consider a set Pnt of $3f{+}1$ **participants**.

- Call a subset $Q \subseteq \mathsf{Pnt}$ a **quorum** when $\#Q \geq 2f{+}1$;
- call Q a **coquorum** when $\#Q \leq f$; and
- call Q a **contraquorum** or **blocking set** when $\#Q \geq f{+}1$.

A protocol to hold a ballot to choose between *true* **t** and *false* **f** proceeds as follows:

(1) Each participant $p \in \mathsf{Pnt}$ **broadcasts** to all participants a **vote** message carrying a payload **f** or **t**.
(2) Then:
   (a) If p receives vote-**t** messages from a quorum of participants, then p deems that the ballot has succeeded and p **observes t**.
   (b) If p receives vote-**f** messages from a quorum of participants, then p deems that the ballot has succeeded and p **observes f**.
   (c) If p receives no quorum of messages voting for **t** and no quorum of messages voting for **f**, then p deems that the ballot has failed, and p observes no value.

Remark 2.1.2 We need to choose our failure assumptions:

- Do messages always arrive?
- Can any participants crash?
- Do all participants follow the algorithm, or could some be dishonest/faulty/byzantine/hostile (= not follow the algorithm)?

There is no right or wrong here: we just need to be clear about our assumptions.[7]

---

[6] We defer a treatment of quantifiers and formal logical syntax to the case of Bracha Broadcast.

[7] For convenient reference, I include here a brief survey of some of the terminology used in the distributed systems and blockchain literature.
Call a participant in a distributed algorithm *correct* or *honest* when they follow the protocol of the algorithm. These are synonymous, but the former suggests that we view the participant as a machine, whereas the latter suggests a participant with agency to choose whether to misbehave (e.g. in a blockchain system). Call a participant that does not follow the protocol *faulty* (for machines) or *byzantine* or *adversarial* or *hostile* (for participants with agency).
Types of fault include *crash fault* (stops sending messages), *byzantine fault* (may deviate arbitrarily from the proto-

$p \wedge q$	t	b	f
t	t	b	f
b	b	b	f
f	f	f	f

$p \vee q$	t	b	f
t	t	t	t
b	t	b	b
f	t	b	f

$p \oplus q$	t	b	f
t	f	b	t
b	b	b	b
f	t	b	f

$p \to q$	t	b	f
t	t	b	f
b	t	b	b
f	t	t	t

$p \rightharpoonup q$	t	b	f
t	t	f	f
b	t	b	b
f	t	t	t

$\neg p$	
t	f
b	b
f	t

$\mathsf{T}p$	
t	t
b	f
f	f

$\mathsf{F}p$	
t	f
b	f
f	t

$\mathsf{TB}p$	
t	t
b	t
f	f

$\mathsf{TF}p$	
t	t
b	f
f	t

$\mathsf{B}p$	
t	f
b	t
f	f

Vertical axis above indicates values of p; horizontal axis (if nontrivial) indicates values of q.

Fig. 1. Truth-tables for three-valued connectives on $\mathbf{3} = \{\mathbf{t}, \mathbf{b}, \mathbf{f}\}$ (Definition 2.2.1(2))

For this exercise our failure assumptions will be as follows:

- message-passing is reliable (messages always arrive; they do not get lost or delayed);
- participants never crash;
- participants always send messages when they should; however, some participants may be dishonest and send vote-**t** messages to some participants and vote-**f** messages to others, contrary to the protocol which specifies they should broadcast the *same* vote value to *all* participants.

This might lead to a situation in which honest participants disagree about which vote won the ballot. We want to make sure that this is impossible; or to be more precise, we want to know what conditions are sufficient to ensure Agreement (this property is also called Consistency):

all honest participants observe the same value, if they observe any value at all.

We can prove Agreement/Consistency under an assumption that there is a quorum Q of honest participants, i.e. that there are at most f dishonest participants: Observe by a counting argument that any two quorums $Q_1, Q_2 \subseteq \mathsf{Pnt}$ of votes (each of which contains at least $2f{+}1$ participants) must intersect Q on at least one participant $q \in Q \cap Q_1 \cap Q_2$. Since $q \in Q$, it is honest. This means it is impossible for a participant p to observe $\#Q_1 \geq 2f{+}1$ vote messages for **t**, while at the same time *another* participant p' observes $\#Q_2 \geq 2f{+}1$ vote messages for **f**, because one participant is honest and votes either for **t** or **f**.

We now show how to declaratively model the protocol and the proof of Agreement. First, we will need two ingredients: a notion of *truth*, which we define in Subsection 2.2; and a notion of *quorum*, which we define in Subsection 2.3.

2.2 Three-valued logic

Our notion of truth will be three-valued:

Definition 2.2.1

(1) Let $\mathbf{3} = (\mathbf{f} < \mathbf{b} < \mathbf{t})$ be a lattice of **truth-values** with elements
- **f** (false), which is less than
- **b** (Both/Byzantine/Broken/amBivalent), which is less than
- **t** (true).

(2) In Figure 1 we define:
- a unary connective $\neg$ (**negation**), and

---

col), or *omission fault* (observes the protocol when live, but might choose to fake a crash).
In real life, it is also possible to play timing games (e.g. acting at the last possible moment). These are all 'honest' in a technical sense, though not necessarily in a social or legal sense; e.g. *front running* in trading is illegal. That is out of scope for this paper.

$$tv \vee tv' = \neg(\neg tv \wedge \neg tv') \qquad tv \oplus tv' = (tv \wedge \neg tv') \vee (\neg tv \wedge tv')$$
$$tv \to tv' = (\neg tv) \vee tv' \qquad tv \rightharpoonup tv' = tv \to \mathsf{T}\, tv'$$
$$\mathsf{F}\, tv = \mathsf{T} \neg tv \qquad \mathsf{TB}\, tv = \neg \mathsf{F}\, tv$$
$$\mathsf{TF}\, tv = \mathsf{T}\, tv \vee \mathsf{F}\, tv \qquad \mathsf{B}\, tv = \neg \mathsf{TF}\, tv$$

Fig. 2. Connectives derived from $\neg$, $\wedge$, and T (Remark 2.2.2)

- binary connectives $\wedge$ (**conjunction**), $\vee$ (**disjunction**), $\oplus$ (**exclusive-or**), $\to$ (**weak implication**), and $\rightharpoonup$ (**strong implication**),
- modalities T, B, F, TB, and TF.[8]

(3) Suppose X is a set. We may lift connectives from $\mathbf{3}$ to the function-space $X \to \mathbf{3}$ in the natural **pointwise** definition. Thus: if $f, f' : X \to \mathbf{3}$ then we may use notation as follows:

$$\neg f = \lambda x. \neg f(x)$$
$$f \wedge f' = \lambda x. f(x) \wedge f'(x)$$
$$f \vee f' = \lambda x. f(x) \vee f'(x)$$

(4) If $tv \in \mathbf{3}$ then write $\vDash tv$ for the judgement '$tv \in \{\mathbf{t}, \mathbf{b}\}$' and read this as tv is **valid**.[9] Write $\nvDash tv$ for '$tv = \mathbf{f}$' (this happens precisely when $\neg(\vDash tv)$) and read this as tv is **invalid** or is **false**.

Remark 2.2.2 Note that the connectives in Definition 2.2.1(2) and Figure 1 are not minimal. We can express them all from $\neg$, $\wedge$, and T as per Figure 2.

The implicational structure of $\mathbf{3}$ is rich (it supports sixteen implications [6, note 5, page 22]). For us, the weak implication $\to$ and strong implication $\rightharpoonup$ will suffice, and we have:

Proposition 2.2.3 *For truth-values $tv, tv' \in \mathbf{3}$, we have:*

(1) $\vDash \mathbf{t}$ *and* $\vDash \mathbf{b}$ *and* $\nvDash \mathbf{f}$.
(2) $\vDash tv \vee tv'$ *if and only if* $\vDash tv \vee \vDash tv'$, *and*
$\vDash tv \wedge tv'$ *if and only if* $\vDash tv \wedge \vDash tv'$.
(3) $(tv \to tv') = (\neg tv \vee tv')$ *and* $(tv \rightharpoonup tv') = (tv \to \mathsf{T}\, tv')$.
(4) $\vDash tv \to tv'$ *if and only if* $tv = \mathbf{t}$ *implies* $tv' \in \{\mathbf{t}, \mathbf{b}\}$. *We call this* ***weak modus ponens****.*
(5) $\vDash tv \rightharpoonup tv'$ *if and only if* $tv = \mathbf{t}$ *implies* $tv' = \mathbf{t}$. *We call this* ***strong modus ponens****.*
(6) $\vDash tv \vee \neg tv$. *In jargon, validity satisfies* ***excluded middle*** *(p-or-not-p is valid).*
(7) $\vDash tv \wedge \neg tv$ *if and only if* $tv = \mathbf{b}$ *if and only if* $\vDash \mathsf{B}\, tv$.[10]
(8) $\vDash tv$ *if and only if* $\mathsf{TF}\, tv = \mathsf{T}\, tv$.
(9) $tv \leq tv'$ *if and only if* $\neg tv' \leq \neg tv$.

Proof. By inspection of the truth-tables in Figure 1. □

2.3 Semitopologies

We define our universe of participants. This has a topological flavour:

[8] We call $\neg$ a 'connective' and (for example) T a 'modality'. This is just a matter of tradition: structurally, they are both just unary operators.
[9] There is a standard distinction in logic between a truth-value being *true* (which means: it is equal to $\mathbf{t}$) and a logical judgement being *valid* (which may mean 'a true assertion about a model' or 'it is derivable in some derivation system' or 'it evaluates to true in a given context'). In two-valued logic this distinction is less prominent, because a predicate may be judged to be valid when it evaluates to $\mathbf{t}$ (possibly in the context of some valuation and/or interpretation). But here, we have three truth-values, and a predicate will be judged to be valid when it evaluates to $\mathbf{t}$ *or* $\mathbf{b}$.
[10] This means that our notion of validity is **paraconsistent**, so ϕ and not-ϕ may both be valid ([6], or see the nice survey in [27]). Note that in our setting ϕ and not-ϕ cannot be simultaneously *true*, since $\neg\mathbf{t} = \mathbf{f}$.

$$\begin{array}{rlrl}
(\mathsf{Observe?}) & observe(p) \to \boxdot vote & (\mathsf{Observe\neg?}) & \neg observe(p) \to \boxdot \neg vote \\
(\mathsf{Observe!}) & \boxdot vote \rightharpoonup observe(p) & (\mathsf{Observe\neg!}) & \boxdot \neg vote \rightharpoonup \neg observe(p) \\
(\mathsf{Correct}) & \boxdot(\mathsf{TF} \circ vote) & (\mathsf{3twined}) & (\boxdot f \wedge \boxdot f') \leq \diamondsuit(f \wedge f')
\end{array}$$

Above: $\circ$ denotes function composition; f and f' range over arbitrary functions in $\mathsf{Pnt} \to \mathbf{3}$ and p ranges over elements of Pnt. $\leq$ refers to the lattice ordering on $\mathbf{3}$, namely $\mathbf{f} < \mathbf{b} < \mathbf{t}$. $\boxdot$ and $\diamondsuit$ have type $(\mathsf{Pnt} \to \mathbf{3}) \to \mathbf{3}$ here, which is why *observe* is applied to p above and *vote* is not.
$vote(p) = \mathbf{t}$ means 'p voted for $\mathbf{t}$'; $vote(p) = \mathbf{f}$ means 'p voted for $\mathbf{f}$'; and $vote(p) = \mathbf{b}$ means 'p sent conflicted messages about its vote'. Similarly for *observe*.

Fig. 3. Specification of a simple voting protocol (Definition 2.5.1)

$$\begin{array}{ll}
\Box f = \bigwedge_{p \in \mathsf{Pnt}} f(p) & \diamondsuit f = \bigvee_{p \in \mathsf{Pnt}} f(p) \\
\boxdot f = \bigvee_{O \in \mathsf{Opn}_{\neq \varnothing}} \bigwedge_{p \in O} f(p) & \diamondsuit\!\!\cdot f = \bigwedge_{O \in \mathsf{Opn}_{\neq \varnothing}} \bigvee_{p \in O} f(p).
\end{array}$$

Above, $\Box, \diamondsuit, \boxdot, \diamondsuit\!\!\cdot : (\mathsf{Pnt} \to \mathbf{3}) \to \mathbf{3}$ and we read $\Box f$ as *everywhere-f*, $\diamondsuit f$ as *somewhere-f*, $\boxdot f$ as *quorum-f*, and $\diamondsuit\!\!\cdot f$ as *contraquorum-f*.

Fig. 4. Modalities (Definition 2.3.2)

$$\begin{array}{ll}
\neg(f \wedge f') = (\neg f \vee \neg f') & \neg(f \vee f') = (\neg f \wedge \neg f') \\
\neg\diamondsuit(\neg f) = \Box f & \neg\Box(\neg f) = \diamondsuit f \\
\neg\diamondsuit\!\!\cdot(\neg f) = \boxdot f & \neg\boxdot(\neg f) = \diamondsuit\!\!\cdot f
\end{array}$$

Above: $f, f' : \mathsf{Pnt} \to \mathbf{3}$ and $\wedge, \vee : (\mathbf{3} \to \mathbf{3})^2 \to (\mathbf{3} \to \mathbf{3})$ are defined pointwise as per Definition 2.2.1(3); $\Box, \diamondsuit, \boxdot, \diamondsuit\!\!\cdot : (\mathsf{Pnt} \to \mathbf{3}) \to \mathbf{3}$; innermost $\neg$ has type $(\mathsf{Pnt} \to \mathbf{3}) \to (\mathsf{Pnt} \to \mathbf{3})$ (it is defined on f and f' pointwise), and outermost $\neg$ has type $\mathbf{3} \to \mathbf{3}$.

Fig. 5. De Morgan dualities (Lemma 2.3.3)

Definition 2.3.1 A **semitopology** [12,13] is a pair $(\mathsf{Pnt}, \mathsf{Opn})$ of

(1) a nonempty [11] set of **points** Pnt – which we may call **participants**, because we may think of them as participants running a distributed algorithm – and
(2) **open sets** $\mathsf{Opn} \subseteq \mathit{powerset}(\mathsf{Pnt})$, such that $\mathsf{Pnt} \in \mathsf{Opn}$ and Opn is closed under arbitrary (possibly empty) unions: if $\mathcal{O} \subseteq \mathsf{Opn}$ then $\bigcup \mathcal{O} \in \mathsf{Opn}$.

Write $\mathsf{Opn}_{\neq \varnothing} = \{O \in \mathsf{Opn} \mid O \neq \varnothing\}$. We may call $O \in \mathsf{Opn}_{\neq \varnothing}$ a **quorum**. Note that we assumed $\mathsf{Pnt} \neq \varnothing$ so that also $\mathsf{Opn}_{\neq \varnothing} \neq \varnothing$ (i.e. some quorum exists).

A semitopology naturally gives rise to a modal logical structure, as follows:

Definition 2.3.2 Suppose $(\mathsf{Pnt}, \mathsf{Opn})$ is a semitopology and $f : \mathsf{Pnt} \to \mathbf{3}$ is a function, which we can think of as a unary predicate (a three-valued one, taking truth-values in $\mathbf{3}$). Define functions

$$\Box, \diamondsuit, \boxdot, \diamondsuit\!\!\cdot : (\mathsf{Pnt} \to \mathbf{3}) \to \mathbf{3}$$

— where we read $\Box f$ as '**everywhere-f**', $\diamondsuit f$ as '**somewhere-f**', $\boxdot f$ as '**quorum-f**', and $\diamondsuit\!\!\cdot f$ as '**contraquorum-f**' — as per Figure 4.

Lemma 2.3.3 *We note some* de Morgan *dualities:*

(1) *Suppose* Pnt *is a set and* $f, f' : \mathsf{Pnt} \to \mathbf{3}$. *Then we have the equivalences in Figure 5.*
(2) *Suppose* $tv \in \mathbf{3}$. *Then* $\neg\mathsf{T}\neg tv = \mathsf{TB}\, tv$ *and* $\neg\mathsf{TB}\neg tv = \mathsf{T}\, tv$.

Proof. By routine arguments on lattices and truth-tables. □

[11] Cf. Remark 2.3.8.

Remark 2.3.4 Semitopologies [12,13] were designed to generalise the quorum systems like Example 2.1.1 (expositions are in [24,3]): *quorum* corresponds to *nonempty open set*; *coquorum* (the set complement to a quorum) corresponds to *non-total closed set*; and *contraquorum* or *blocking set* (a set that intersects every quorum) corresponds to *dense set*.

There are some nice benefits to working with a semitopology as per Definition 2.3.1, rather than with a concrete structure as per Example 2.1.1:

(1) '$O \in \mathsf{Opn}_{\neq\varnothing}$' is shorter, and more mathematically rich, than 'a set O having $2f+1$ elements'.
(2) It notes something that is arguably obvious in retrospect, but which does not appear to have been noted before semitopologies: that certain very basic concepts in the distributed systems literature — including quorums and blocking sets — are topological in nature — corresponding e.g. to nonempty open sets and dense sets. [12]
(3) As Definition 2.3.2 and Lemma 2.3.3 suggest, and the rest of this paper substantiates, we can exploit the connection between topology and logic to reason axiomatically on distributed algorithms.

Remark 2.3.5 T and TB commute with all monotone connectives: for $\mathsf{M} \in \{\mathsf{T}, \mathsf{TB}\}$ and $tv, tv' \in \mathbf{3}$ we have $\mathsf{M}(tv \wedge tv') = ((\mathsf{M}tv) \wedge \mathsf{M}tv')$ and similarly for $\vee$, $\Diamond$, $\Box$, $\boxdot$, and $\diamondsuit$.

The reader can also easily check facts like $\mathsf{T}\,\mathsf{TB}\,tv = \mathsf{T}\,tv = \mathsf{TB}\,\mathsf{T}\,tv$, and $\mathsf{T}\,\mathsf{B}\,tv = \mathsf{B}\,tv$, and so on.

We may rewrite using these properties without comment henceforth.

We warm up with an easy lemma that connects $\Box$ with $\boxdot$:

Lemma 2.3.6 *Suppose* $(\mathsf{Pnt}, \mathsf{Opn})$ *is a semitopology and* $f, f' : \mathsf{Pnt} \to \mathbf{3}$. *Then*

$$(\Box f \wedge \boxdot f') \leq \boxdot(f \wedge f').$$

(Recall that $\mathbf{f} < \mathbf{b} < \mathbf{t}$ *in* $\mathbf{3}$ *as a lattice. The* $\leq$ *above refers to this lattice ordering.)*

As corollaries:

(1) $\vDash \Box f \wedge \boxdot f'$ *implies* $\vDash \boxdot(f \wedge f')$.
(2) $\vDash \Box f \wedge \boxdot(\mathsf{TF} \circ f)$ *implies* $\vDash \mathsf{T}\boxdot f$.

Proof. By routine lattice calculations from Definition 2.3.2. Or, using topological language: if every $p \in \mathsf{Pnt}$ satisfies $f(p)$, and if there exists $O' \in \mathsf{Opn}_{\neq\varnothing}$ such that every $p \in O'$ satisfies $f'(p)$, then clearly every $p \in O' \in \mathsf{Opn}_{\neq\varnothing}$ satisfies $f(p) \wedge f'(p)$. The first corollary follows by routine reasoning, and the second follows taking $f' = \mathsf{TF} \circ f$. □

A characteristic fact connecting $\boxdot$, $\diamondsuit$, and $\Diamond$ is this:

Lemma 2.3.7 *Suppose* $(\mathsf{Pnt}, \mathsf{Opn})$ *is a semitopology and* $f, f' : \mathsf{Pnt} \to \mathbf{3}$. *Then*

$$(\boxdot f \wedge \diamondsuit f') \leq \Diamond(f \wedge f').$$

As corollaries:

(1) $\vDash \boxdot f \wedge \diamondsuit f'$ *implies* $\vDash \Diamond(f \wedge f')$.
(2) $\vDash \diamondsuit f'$ *implies* $\vDash \Diamond f'$.
(3) $\vDash \boxdot(\mathsf{TF} \circ f')$ *and* $\vDash \diamondsuit f'$ *implies* $\vDash \mathsf{T}\Diamond f'$ *(equivalently:* $\vDash \Diamond\mathsf{T}\,f'$*).*

Proof. By routine lattice calculations from Definition 2.3.2. Or, using topological language: if we have an open set of fs and a dense set of f's, then there must be some point with both f and f'. Part 1 follows; part 2 follows from part 1 setting $f = \lambda p.\mathbf{t}$ and recalling from Definition 2.3.1 that $\mathsf{Opn}_{\neq\varnothing} \neq \varnothing$ so that the universal quantification in $\diamondsuit f$ (Definition 2.3.2) cannot be vacuously satisfied; and part 3 follows from part 1 taking $f = \mathsf{TF} \circ f'$. □

[12] While the observation 'quorum system'='nonempty open set of a semitopology' appears to be novel, there is pedigree for making other kinds of connections between (algebraic) topology and distributed systems. We discuss this, with references, in the Conclusions (Section 6).

Remark 2.3.8 In Definition 2.3.1(1) we assume that the set of points is nonempty, disallowing the *empty semitopology* $(\varnothing, \{\varnothing\})$. In [12, Definition 1.2.2] and [13, Definition 1.1.2] we do not impose this condition, i.e. we allow the empty semitopology. There are two reasons for this difference:

(1) In [12,13] we are studying semitopologies *in the abstract*, so we want to be as general as possible. Here, we use semitopologies specifically to model quorum systems.

A distributed system with no participants is just not interesting for our purposes here; there is literally nothing to talk about and in particular this semitopology would have no quorums (nonempty open sets). In context, this possibility would be pathological and possibly confusing, so we outlaw it by assuming that the set of participants is nonempty.

(2) Related to the previous point, but more technically, admitting the empty semitopology would mean that $\diamondsuit(\lambda p.\mathbf{f})$ could equal $\mathbf{t}$. Mathematically that is perfectly respectable, but it would slightly complicate the statement of Lemma 2.3.7(2); we would need to explicitly add a condition 'for nonempty semitopology'. [13] It is simpler and neater to just insist that all semitopologies are nonempty.

Remark 2.3.9 In Definition 2.3.1 we define semitopologies, which (as per Remark 2.3.4) abstract the notion of *quorum system*. Definition 2.3.1(2) insists that the open sets of a semitopology be closed under unions, but in fact we will not use that condition in the rest of this paper! It is there because it is inherited from the ambient semitopological mathematics; for the purposes of *this* paper, we could drop it and the mathematics would work just as well.

To couch this in fancy jargon: we could work with just a basis of the semitopology, where a **basis** of a semitopology $(\mathsf{Pnt}, \mathsf{Opn})$ is a subset $\mathcal{O} \subseteq \mathsf{Opn}$ that generates all of Opn by arbitrary set unions.

However, in a deeper sense closure under unions is very much still present:

(1) In the literature, quorum systems used for voting, Bracha Broadcast, and Crusader Agreement are traditionally what we can call *threshold-based*; meaning that they have the form 'a quorum is a set with more than t elements' for some threshold t, where usually $t = n * f + 1$ for parameters n and f (see Example 2.1.1, Subsection 2.4, and Example 2.4.4(1)).

Threshold-based quorum systems *are* closed under arbitrary unions. Indeed, they satisfy the stronger property of being *up-closed* (if $O \in \mathsf{Opn}$ and $O \subseteq O' \subseteq \mathsf{Pnt}$ then $O' \in \mathsf{Opn}$). Thus the semitopologies of Definition 2.3.1 as written are a *de facto* generalisation of the quorum systems as used in the literature relevant to the examples in this paper.

(2) Related to the previous point, the form of our axioms and correctness properties will be such that they are functionally insensitive to the difference between a full semitopology and a basis of that semitopology. Specifically, all conditions on the size of a set of participants are positive, and we will not see any negative condition in an axiom or correctness property that a set of participants should 'not be too big'. [14] In this sense, the reason that we do not use the closure under unions condition from Definition 2.3.1(2) explicitly, is because it is so fundamental that it is actually structurally built in to the form of the axioms and correctness properties.

This positivity criterion may be a useful static (i.e. syntactic) criterion for what makes a good axiom system, but we leave thinking about that to future work (see also Subsection 6.6.5).

To sum up, closure under unions is not explicitly used in this paper, but it lurks in the background: in the form of concrete quorum systems used in the literature; in the ambient semitopological metatheory; in the fact that threshold-based quorum systems satisfy this property anyway; *and* also (I would argue) it is implicit in the sense that our axioms and correctness properties are constituted precisely to work as well with a full semitopology as with a basis of that semitopology, and this seems likely to be an important well-behavedness criterion for protocols.

[13] Thanks to Jan Mas Rovira for noting this subtlety.

[14] This is also arguably part of why quorum systems in the literature are up-closed; since up-closure is functionally invisible to the protocols, a threshold quorum system is perfectly sufficient and it is unintuitive, and not worth the bother, to insist on quorums having 'exactly t elements'.

2.4 3-twined semitopologies

2.4.1 Definition, discussion, and examples

Definition 2.4.1 For $n \geq 0$, call a semitopology $(\mathsf{Pnt}, \mathsf{Opn})$ **n-twined** when any n nonempty open sets have a nonempty intersection [17, Section 5]. That is: if $O_1, \ldots, O_n \in \mathsf{Opn}_{\neq\varnothing}$ then $\bigcap_{1\leq i\leq n} O_i \neq \varnothing$.

In this paper we will care most about 3-twined semitopologies; semitopologies such that any three nonempty open sets have a nonempty intersection.

Remark 2.4.2 Indexing the open sets from 1 in Definition 2.4.1 even though we only insist that $n \geq 0$, is not a typo. By convention, $\bigcap \varnothing = \mathsf{Pnt}$, so a semitopology is 0-twined just when Pnt is nonempty; as per Remark 2.3.8 Pnt is assumed nonempty, so every (nonempty) semitopology is 0-twined. The reader can also check that every semitopology is 1-twined.

So the n-twined condition starts getting more structured from 2-twined onwards.

The terminology '3-twined' follows the semitopologies literature: it generalises the notion of *intertwined* space considered in [12,13] (intertwined = 2-twined) and see also the Section on n-twined semitopologies in [17]. However, the general concept is well-known in the literature, albeit not phrased systematically and in (semi)topological terms. For example, being 3-twined is called the Q^3 property in [3, Definition 2].

Remark 2.4.3 We take a moment to motivate 3-twinedness: why is this of such interest to distributed systems? Note that:

- A typical step in a distributed protocol has the general form:
  "Wait until you see a quorum Q (= nonempty open set) of participants perform some action A, and then perform action B".
  For example, the ($\mathsf{BrDeliver}?/!$) and ($\mathsf{BrReady}?/!$) axiom-pairs in Figure 10 have *precisely* this form; and other axioms are variations on it.
- A typical correctness assumption is that there exists a quorum C of correct (= honest / non-faulty) participants (see for example ($\mathsf{BrCorrect}$) and ($\mathsf{BrCorrect}'$)).

Now consider two participants p and p' running some protocol X, and suppose that:

(1) p sees a quorum Q of participants perform action A and so according to X is ready to perform action B, and
(2) p' sees a quorum Q' of participants perform action A' and so according to X is ready to perform action B'.

If the semitopology is 3-twined, then there exists some $q \in Q \cap Q' \cap C$. This means that p and p' know that there exists some q who is honest and has performed actions A and A'; they do not know who q is, but they know that q exists.

This can give p and p' confidence to progress to perform actions B and B' respectively, safe in the knowledge that A and A' are compatible, in the sense that there was at least one honest participant who was willing to do both.

If this sounds trivial, it is not. For example A might be 'vote for value v' and A' might be 'vote for v'', and perhaps the protocol insists that (honest) participants should only vote for one possibility. Then in this case, p and p' know that $v = v'$ — even though they do not know who the honest participants are, and even though p and p' may not even know about each other. In a bit more detail, 3-twinedness gives us this intuitive property:

every honest participant knows that some honest participant exists in any pairwise quorum intersection,

and as per our example above, this in and of itself may be enough confidence for individual participants like p and p' (even without speaking to one another) to progress with confidence just on the basis of observing a relevant quorum.

This in a nutshell is the content of Theorem 2.4.5, Corollary 2.4.6, and axiom ($\mathsf{3twined}$) in Figure 3, and furthermore, Remark 2.4.8 proves that this intuition is (in a sense made mathematical in that Remark) not only sound but also complete.

The semitopology in Example 2.1.1 is the canonical example of a 3-twined semitopology, but trivially an n-twined semitopology is n'-twined for every $n' \leq n$, and examples of n-twined semitopologies for

arbitrary n are very easy to generate, for example as follows:

Example 2.4.4 We indicate two schemas by which the reader can generate n-twined semitopologies. For $n \geq 3$, any of these could be used in the development to follow, with no change to the proofs (since all we will require is for our semitopology to be 3-twined):

(1) Suppose $f \geq 0$ and $n \geq 1$ and let Pnt be any set with cardinality $n{*}f{+}1$. Endow Pnt with a semitopological structure by letting Opn be the set containing the empty set, and containing any set with cardinality at least $(n\text{-}1){*}f+1$. Then the reader can easily check that $(\mathsf{Pnt}, \mathsf{Opn})$ is an n-twined semitopology.

(2) Fix a logical theory Φ in first-order logic [15] and fix a first-order logic predicate $\phi(x, y)$ with (at most) two free variables x and y, and assume the following property:

$$\Phi \vdash n\text{-}twined_\phi \quad \text{where} \quad n\text{-}twined_\phi = \forall y_1, \ldots, y_n.\exists x.(\phi(x, y_1) \wedge \cdots \wedge \phi(x, y_n)).$$

Here $\vdash$ is the standard symbol for derivability, so $\Phi \vdash n\text{-}twined_\phi$ means that we can prove that for every $y_1, \ldots, y_n$ there exists x such that $\phi(x, y_1), \ldots, \phi(x, y_n)$ are all simultaneously true. By Soundness of first-order logic, it follows that $n\text{-}twined_\phi$ is valid in every model of Φ.

Now fix some model M of Φ. This means that every axiom in Φ is valid in M, so that by Soundness and our assumption that $\Phi \vdash n\text{-}twined_\phi$, also $M \vDash n\text{-}twined_\phi$. Here $\vDash$ is the standard symbol for model-theoretic validity, so $M \vDash n\text{-}twined_\phi$ means that $n\text{-}twined_\phi$ *is actually valid* of M.

Give (the underlying set of) M a semitopological structure by letting open sets be the empty set, the whole set, or arbitrary unions of sets of the form $\{x \in M \mid M \vDash \phi(x, y)\}$ where $y \in M$. By construction this makes M into an n-twined semitopology, since $n\text{-}twined_\phi$ ensures that any n nonempty open sets *actually do* intersect at some x.

Now this example might seem like just an absurdly abstract way to generate n-twined semitopologies, and in a sense it is, but in another sense it is much more than that: it is *the* way to generate n-twined semitopologies.

A choice of Φ and ϕ such that $\Phi \vdash n\text{-}twined_\phi$, specifies a class of n-twined semitopologies, and conversely, to the extent that any practical construction of n-twined semitopologies can be expressed in first-order logic, *every* such construction can be so expressed. [16]

So the difference between this example and (say) the grid quorum systems in [31, Section 4] is just how specific we are about picking Φ and ϕ.

Because we are using an axiomatic approach, we do not need to be any more specific about the underlying model than the axioms require. In what follows, we can and we will just reason assuming an arbitrary 3-twined semitopology. This is all the detail that the axioms require, and so that will be all that we need.

2.4.2 The fundamental logical characterisation of being 3-twined, and its corollaries

Reading Definition 2.4.1 literally, we can render it as $(\boxdot f \wedge \boxdot f' \wedge \boxdot f'') \leq \diamondsuit(f \wedge f' \wedge f'')$, for any $f, f', f'' : \mathsf{Pnt}{\to}\mathbf{3}$. However, this is not always the most convenient logical form of the property. Arguably, the fundamental logical property of a 3-twined semitopology is this:

Theorem 2.4.5 *Suppose* $(\mathsf{Pnt}, \mathsf{Opn})$ *is 3-twined and* $f, f' : \mathsf{Pnt}{\to}\mathbf{3}$. *Then*

$$(\boxdot f \wedge \boxdot f') \leq \diamondsuit(f \wedge f').$$

As a corollary

$$\vDash \boxdot f \wedge \boxdot f' \quad \textit{implies} \quad \vDash \diamondsuit(f \wedge f').$$

(A converse to this result also holds: see Remark 2.4.8.)

Proof. $\boxdot f \wedge \boxdot f' = \mathbf{t}$ implies that there exist $O, O' \in \mathsf{Opn}_{\neq\varnothing}$ on which f and f' respectively take the value $\mathbf{t}$. By the 3-twined property, $O \cap O'$ intersects every other $O'' \in \mathsf{Opn}_{\neq\varnothing}$, thus $\diamondsuit(f \wedge f') = \mathbf{t}$.

[15] A *logical theory* is a set of closed predicates (having no free variables; they can mention bound variables), which we call *axioms*.

[16] In principle we could imagine using more powerful logics, but every quorum system that I have seen in the literature so far is clearly a first-order construction.

$\boxdot f \wedge \boxdot f' = \mathbf{b}$ implies that there exist $O, O' \in \mathsf{Opn}_{\neq\varnothing}$ on which f and f' respectively take values in $\{\mathbf{t}, \mathbf{b}\}$. By the 3-twined property $O \cap O'$ intersects every other $O'' \in \mathsf{Opn}_{\neq\varnothing}$, thus $\diamondsuit(f \wedge f') \in \{\mathbf{t}, \mathbf{b}\}$. □

Corollary 2.4.6 is a (rather elegant) dual rephrasing of Theorem 2.4.5 and will be useful later:

Corollary 2.4.6 Suppose $(\mathsf{Pnt}, \mathsf{Opn})$ is 3-twined and $f, f' : \mathsf{Pnt} \to \mathbf{3}$. Then

$$\boxdot(f \vee f') \leq (\diamondsuit f \vee \diamondsuit f').$$

As a corollary,

$$\vDash \boxdot(f \vee f') \quad \text{implies} \quad \vDash \diamondsuit f \vee \diamondsuit f'.$$

Proof. By Theorem 2.4.5 $(\boxdot\neg f \wedge \boxdot\neg f') \leq \diamondsuit(\neg f \wedge \neg f')$. By Proposition 2.2.3(9) $\neg\diamondsuit(\neg f \wedge \neg f') \leq \neg(\boxdot\neg f \wedge \boxdot\neg f')$. Simplifying using Lemma 2.3.3(1) we obtain $\boxdot(f \vee f') \leq (\diamondsuit f \vee \diamondsuit f')$ as required. □

Remark 2.4.7 It might be helpful to sum up some highlights of the maths so far. Suppose $(\mathsf{Pnt}, \mathsf{Opn})$ is a semitopology and $f, f' : \mathsf{Pnt} \to \mathbf{3}$.

Suppose further that $\vDash \boxdot(\mathsf{TF} \circ f)$, meaning that f is correct (= $\mathbf{t}$ or $\mathbf{f}$, not $\mathbf{b}$) on a quorum (= nonempty open set) of participants. (It is a typical failure assumption in the distributed systems literature that there exists a quorum of correct participants.)

Then we have seen that:

(1) $\vDash \Box f$ implies $\vDash \mathsf{T}\boxdot f$, by Lemma 2.3.6(2).
(2) If $(\mathsf{Pnt}, \mathsf{Opn})$ is 3-twined then $\vDash \boxdot f$ implies $\vDash \diamondsuit(\mathsf{T} \circ f)$ and equivalently $\vDash \mathsf{T}\diamondsuit f$, by Theorem 2.4.5.
(3) $\vDash \diamondsuit f$ implies $\vDash \Diamond(\mathsf{T} \circ f)$ and equivalently $\vDash \mathsf{T}\Diamond f$, by Lemma 2.3.7(3).
(4) $\vDash \boxdot f \wedge \diamondsuit(\mathsf{T} \circ f)$ implies $\vDash \mathsf{T}\Diamond f$, from Lemma 2.3.7(1) (taking f' in that Lemma to be $\mathsf{T} \circ f$).
(5) If $(\mathsf{Pnt}, \mathsf{Opn})$ is 3-twined then $\vDash \boxdot f \wedge \boxdot f'$ implies $\vDash \diamondsuit(f \wedge f')$ and $\vDash \boxdot(f \vee f')$ implies $\vDash \diamondsuit f \vee \diamondsuit f'$, by Theorem 2.4.5 and Corollary 2.4.6.

Remark 2.4.8 For completeness we mention that the reverse implication in Theorem 2.4.5 also holds (though we will not need it here): if for every $f, f' : \mathsf{Pnt} \to \mathbf{3}$ we have $(\boxdot f \wedge \boxdot f') \leq \diamondsuit(f \wedge f')$, then $(\mathsf{Pnt}, \mathsf{Opn})$ is 3-twined.

The argument is as follows: Suppose we have three nonempty open sets $O, O', O'' \in \mathsf{Opn}_{\neq\varnothing}$. Following Definition 2.4.1 we must show that $O \cap O' \cap O'' \neq \varnothing$, and our assumption is that for any functions $f, f' : \mathsf{Pnt} \to \mathbf{3}$ we have $(\boxdot f \wedge \boxdot f') \leq \diamondsuit(f \wedge f')$. We choose f to map $p \in O$ to $\mathbf{t} \in \mathbf{3}$ and $p \notin O$ to $\mathbf{b} \in \mathbf{3}$, and we choose f' to map $p \in O'$ to $\mathbf{t}$ and $p \notin O'$ to $\mathbf{b}$.[17] By the construction of $\boxdot$ in Definition 2.3.2, $\boxdot f = \mathbf{t} = \boxdot f'$, so by our assumption $\diamondsuit(f \wedge f') = \mathbf{t}$. By the construction of $\diamondsuit$ in Definition 2.3.2, it must be that $O \cap O' \cap O'''$ is nonempty for *every* $O''' \in \mathsf{Opn}_{\neq\varnothing}$, and in particular $O \cap O' \cap O''$ is nonempty as required.

2.5 A simple voting protocol

We are now ready to consider our first (very simple) protocol:

Definition 2.5.1 Call a tuple $\mu = ((\mathsf{Pnt}, \mathsf{Opn}), \mathit{vote}, \mathit{observe})$, where

- $(\mathsf{Pnt}, \mathsf{Opn})$ is a(n arbitrary) semitopology and
- $\mathit{vote}, \mathit{observe} : \mathsf{Pnt} \to \mathbf{3}$ are functions,

a **model**. We call a model μ a **model** (of ThyVote) when the axioms in Figure 3 are valid in μ.

Remark 2.5.2 We take a minute to unpack what the notation in Figure 3 is intended to mean:

[17] Why **b** and not **f**? We could just as well use **f**, but the benefit of using **b** is that f and f' are also *continuous* to **3** with its natural semitopological structure as developed in [12,13]. In this paper we do not develop continuity, but in a wider context it is nice to have an 'if-and-only-if' not just with respect to the set of all $f, f' : \mathsf{Pnt} \to \mathbf{3}$, but also specifically for the continuous ones.

(1) $vote(p) = \mathbf{t}$ indicates that p voted for $\mathbf{t}$. $vote(p) = \mathbf{f}$ indicates that p voted for $\mathbf{f}$. $vote(p) = \mathbf{b}$ indicates that p (is hostile because it) sent vote messages for *both* $\mathbf{t}$ and $\mathbf{f}$. [18]
(2) $observe(p) = \mathbf{t}$ indicates that p observed a quorum of vote-$\mathbf{t}$ messages – some of which may be from hostile participants; p cannot know. Similarly for $observe(p) = \mathbf{f}$. $observe(p) = \mathbf{b}$ indicates that p observed no quorum of messages either way.
(3) *vote* and *observe* may look like messages ('vote', 'observe'), but they are functions of type $\mathsf{Pnt} \to \mathbf{3}$ (the reader used to higher-order logic may wish to call them *predicates*).

The intuitive meaning of $vote(p)$ and $observe(p)$ is 'participant p voted/observed during a run of the protocol'. [19] But the mathematical meaning is just applying a function to a $p \in \mathsf{Pnt}$ to get a truth-value in $\mathbf{3}$.

Remark 2.5.3 We discuss how the axioms in Figure 3 correspond to the protocol in Example 2.1.1:

(1) (Observe?) is a **backward rule**: it expresses that *if* a participant p observes $\mathbf{t}$, then p must have received a quorum of vote-$\mathbf{t}$ messages.

We use weak implication $\to$ (not strong implication $\rightharpoonup$) in (Observe?) because a quorum may include some dishonest (hostile) participants who sent dishonest messages; i.e. vote-$\mathbf{t}$ to some participants and vote-$\mathbf{f}$ to others. We represent the combination of vote-$\mathbf{t}$ and vote-$\mathbf{f}$ messages from a hostile participant q by setting $vote(q) = \mathbf{b}$. Thus, if $observe(p) = \mathbf{t}$ it does not follow that $\boxdot vote = \mathbf{t}$; it only follows that $\boxdot vote \in \{\mathbf{t}, \mathbf{b}\}$. This highlights that $vote(p)$ in our declarative model is a single *truth-value*, not a *message* or a set of messages.
(2) (Observe¬?) just repeats (Observe?) for the case that p receives a quorum of vote-$\mathbf{f}$.
(3) (Observe!)/(Observe¬!) are **forward rules**: they express that if an honest quorum votes for $\mathbf{t}/\mathbf{f}$, then every p observes $\mathbf{t}/\mathbf{f}$.

Why did we use strong implication here, not weak implication? We could use either, but we make a design choice: since (in our protocol) participants do not report their *observe* to anyone else, there is no useful sense in which lying would be meaningful. So we use the strong implication.
(4) (Correct) asserts that some quorum of participants are honest: they vote $\mathbf{t}$ or $\mathbf{f}$, and not $\mathbf{b}$.
(5) (3twined) expresses the characteristic intersection property of a 3-twined semitopology, as per Theorem 2.4.5. For this protocol we will use it once, setting $f = vote$ and $f' = \neg vote$.

Remark 2.5.4 At the level of *vote*, $\mathbf{b}$ denotes a hostile participant sending mixed votes to different participants. At the level of *observe*, $\mathbf{b}$ just indicates that a participant has observed no quorum of participants voting for $\mathbf{t}$ or voting for $\mathbf{f}$.

This use of $\mathbf{b}$ to mean two things, depending on the context, is a feature, not a bug. It illustrates a fact about truth-values in $\mathbf{3}$: they are just data, which we – via our choice of axioms and intended interpretation – can interpret as we choose and as is mathematically convenient. [20]

Remark 2.5.5 Note in Figure 3 that (Observe?) has a dual (Observe¬?), and (Observe!) has a dual (Observe¬!) – but (Correct) has no dual axiom (Correct¬). Why is this so? Because (Correct) is self-dual: it is a fact of the truth-tables in Figure 1 that $\mathsf{TF} \circ vote = \mathsf{TF} \circ \neg \circ vote$, so that $\boxdot(\mathsf{TF} \circ vote) = \boxdot(\mathsf{TF} \circ \neg \circ vote)$.

Lemma 2.5.6 $\vDash \Diamond observe \to \boxdot vote$, $\vDash \boxdot vote \rightharpoonup \Box observe$, $\vDash \Diamond \neg observe \to \boxdot \neg vote$ *and* $\vDash \boxdot \neg vote \rightharpoonup \Box \neg observe$.

Proof. The reasoning is very easy and we prove just the first two parts.

(1) By weak modus ponens (Prop. 2.2.3(4)) it suffices to show that $\vDash \mathsf{T} \Diamond observe$ implies $\vDash \boxdot vote$. So suppose $\vDash \mathsf{T} \Diamond observe$. Using the meaning of $\Diamond$ from Definition 2.3.2, there exists $p \in \mathsf{Pnt}$ such

[18] There is no notion here of 'not voted yet', because there is no notion of time! *vote* just records the votes that a participant casts, and our failure assumptions assume that *some* vote is made, as per Remark 2.1.2. Different assumptions would yield different axioms.
[19] Normally we would vote for or observe a value, but in this simple protocol we leave the values out.
[20] An example from real life. If my wife asks me "You didn't forget to put the rubbish out, did you?" and I return a value "Yes", does this mean "Yes, I did forget!" or "Yes, I did put the rubbish out!"? The "Yes" here is raw data; its meaning exists in the (hopefully shared) understanding of the people having the conversation.
In the same way, $\mathbf{b}$ is just data, and is meaningless until we – *we* – use it in axioms and interpret those axioms.

that $observe(p) = \mathbf{t}$, and so $\models \mathsf{T}\, observe(p)$. Then using weak modus ponens and (Observe?) we have $\models \boxdot vote$.

(2) By strong modus ponens (Prop. 2.2.3(5)) and the meaning of $\Box$, it suffices to show that $\models \mathsf{T}\boxdot vote$ implies $\models \mathsf{T}\, observe(p)$ for every $p \in \mathsf{Pnt}$. But this is immediate from (Observe!). □

We can now prove it is impossible for one participant to observe **t** and another to observe **f**:

Proposition 2.5.7 (Agreement) *If μ is a model of* ThyVote *then $\not\models \Diamond \mathsf{T}\, observe \wedge \Diamond \mathsf{T} \neg observe$.*

Proof. Suppose $\models \Diamond \mathsf{T}\, observe \wedge \Diamond \mathsf{T} \neg observe$. Then there exist $p, p' \in \mathsf{Pnt}$ such that $observe(p) = \mathbf{t}$ and $observe(p') = \mathbf{f}$. Using (Observe?) and (Observe¬?) $\models \boxdot vote \wedge \boxdot \neg vote$. By (3twined) $\models \diamondsuit(vote \wedge \neg vote)$, so using Proposition 2.2.3(7) $\models \diamondsuit \mathsf{B}\, vote$. Using (Correct) and Lemma 2.3.7(1) $\models \Diamond(\mathsf{B}\, vote \wedge \mathsf{TF}\, vote)$. But $(\mathsf{B}\, vote \wedge \mathsf{TF}\, vote) = \mathbf{f}$, and $\models \Diamond \mathbf{f}$ is impossible. □

Remark 2.5.8 A more traditional proof for Proposition 2.5.7 would work concretely with the set of quorums from Example 2.1.1, and prove Agreement by counting and calculations (see for example Lemmas 1 to 4 used in the proof of Theorem 1 from [7, page 135]).

Proposition 2.5.7 is not that proof: the semitopology is abstract, except for satisfying the axioms in Figure 3. The reasoning is logical in flavour and follows the structure of the axioms. This is simple, general, and it captures the logical essence of what the concrete counting proofs are actually doing.

Remark 2.5.9 The failure model is encoded in the axioms – typically this shows up in the strength of the implications we use. For example: our failure model in Remark 2.1.2 assumed a reliable network, which is reflected by the use of *strong implication* in the forward rules (Observe!) and (Observe¬!) in Figure 3.

To model an unreliable network, in which messages sent might not arrive, we would weaken (Observe!) to $\boxdot vote \to observe$, and similarly for (Observe¬!). This reflects that a quorum of honest participants vote for **t**, but the network fails to transmit their messages so a participant p (even if honest) may remain unable to observe and so set $observe(p) = \mathbf{b}$.

3 The modal logic

The above concludes our discussion of the simple voting protocol. We are now ready to start studying Bracha Broadcast. We will need two more ingredients:

(1) Our voting example was binary (vote-**t** or vote-**f**). For Bracha Broadcast we assume an arbitrary nonempty set of values Val, over which we will need three-valued quantifiers like unique existence $\exists_{\mathbf{1}} : \mathsf{Val} \to \mathbf{3}$. We study this in Subsection 3.1.

(2) Our voting example worked directly with semantics; we made assertions directly about functions like $vote : \mathsf{Pnt} \to \mathbf{3}$. For Bracha Broadcast it is better to build the syntax and semantics of a (simple) logic; we will have predicate-symbols like echo, which will be interpreted as functions like $\varsigma(\mathsf{echo}) : \mathsf{Pnt} \to \mathsf{Val} \to \mathbf{3}$. We study this in Subsection 3.2.

3.1 (Unique/affine) existence in three-valued logic

Distributed algorithms often require an implementation to make choices, e.g. 'respond only to the first message you receive'. In axioms, we model this using unique existence and affine existence:

Definition 3.1.1 Suppose Val is a set and $f : \mathsf{Val} \to \mathbf{3}$ is a function.

(1) Write $\mathsf{=} : \mathsf{Val} \to \mathsf{Val} \to \mathbf{3}$ for **3-equality** which is such that $v\mathsf{=}v' = \mathbf{t}$ if $v = v'$ and $v\mathsf{=}v' = \mathbf{f}$ if $v \neq v'$.[21]

(2) Define **existence** $\exists f$, **affine existence** $\exists_{\mathbf{01}} f$ (there exist zero or one) and **unique existence** $\exists_{\mathbf{1}} f$ (there exists precisely one)[22] as per Figure 6.

[21] I hope typography will distinguish between $\mathsf{=}$ (the function that returns a truth-value in **3**) and $=$ (equality in the discourse of this paper).

[22] The standard symbol for unique existence is $\exists!$. I am not aware of a standard symbol for affine existence. *Regular expressions* use ? to denote zero or one matches, so I would like to write affine existence as $\exists?$. Unfortunately we are already using ! and ? to name the forwards and backward rules, and I found it visually confusing to also place ! and ? in the logical syntax. So we write $\exists_1$ and $\exists_{01}$ instead.

$$\exists f = \bigvee_{v \in \mathsf{Val}} f(v)$$
$$\exists_{\mathbf{01}} f = \bigwedge_{v,v' \in \mathsf{Val}} (f(v) \wedge f(v')) \to v{=}v'$$
$$\exists_{\mathbf{1}} f = (\exists f) \wedge (\exists_{\mathbf{01}} f)$$

Above: Val is a set and $f : \mathsf{Val} \to \mathbf{3}$ is a function; $=$ on the left is definitional equality defining the LHS to mean the RHS (this $=$ is *not* an operation on $\mathbf{3}$); $\to$ in the clause for $\exists_{\mathbf{01}} f$ is the weak implication from Figure 1, and $\bigvee$ and $\bigwedge$ denote join (least upper bound) and meet (greatest lower bound) in $\mathbf{3}$-the-lattice $\mathbf{f} < \mathbf{b} < \mathbf{t}$ from Definition 2.2.1 (so $\to$, $\bigvee$, $\bigwedge$ *are* operations on $\mathbf{3}$); and $\mathsf{=} : \mathsf{Val} \to \mathsf{Val} \to \mathbf{3}$ in the clause for $\exists_{\mathbf{01}} f$ is the $\mathbf{3}$-equality from Definition 3.1.1(1).

Fig. 6. Existence, affine existence, and unique existence (Definition 3.1.1)

Remark 3.1.2 Note that $\exists$, $\exists_{\mathbf{1}}$, and $\exists_{\mathbf{01}}$ return truth-values in $\mathbf{3}$, whereas the usual Boolean quantifiers – write them $\exists$, $\exists_1$, and $\exists_{01}$ – return truth-values in $\{\bot, \top\}$. So even if '$\exists_{\mathbf{01}}$' looks like '$\exists_{01}$', these are different creatures.

Therefore, continuing the notation of Definition 3.1.1, we spell out cases to check how the Definition works:

(1) If $\exists v \neq v' \in \mathsf{Val}.f(v) = \mathbf{t} = f(v')$, then $\exists_{\mathbf{1}} f = \mathbf{f} = \exists_{\mathbf{01}} f$.
(2) If $\exists_1 v \in \mathsf{Val}.f(v) = \mathbf{t}$, and $\forall v' \in \mathsf{Val}.f(v') \in \{\mathbf{t}, \mathbf{f}\}$, then $\exists_{\mathbf{1}} f = \mathbf{t} = \exists_{\mathbf{01}} f$.
(3) If $\exists_1 v \in \mathsf{Val}.f(v) = \mathbf{t}$, and $\exists v' \in \mathsf{Val}.f(v') = \mathbf{b}$, then $\exists_{\mathbf{1}} f = \mathbf{b} = \exists_{\mathbf{01}} f$.
(4) If $\neg\exists v \in \mathsf{Val}.f(v) = \mathbf{t}$ and $\exists_1 v' \in \mathsf{Val}.f(v') = \mathbf{b}$, then $\exists_{\mathbf{1}} f = \mathbf{b}$ and $\exists_{\mathbf{01}} f = \mathbf{t}$.
(5) If $\neg\exists v \in \mathsf{Val}.f(v) = \mathbf{t}$ and $\exists v, v' \in \mathsf{Val}.v \neq v' \wedge f(v) = \mathbf{b} = f(v')$, then $\exists_{\mathbf{1}} f = \mathbf{b} = \exists_{\mathbf{01}} f$.
If $\forall v \in \mathsf{Val}.f(v) = \mathbf{b}$, then $\exists_{\mathbf{1}} f = \mathbf{b}$ – note that this is not $\mathbf{f}$*. This illustrates that in our three-valued setting, 'unique existence' is not the same as 'unique non-false value'.*
(6) If $\forall v \in \mathsf{Val}.f(v) = \mathbf{f}$, then $\exists_{\mathbf{1}} f = \mathbf{f}$ and $\exists_{\mathbf{01}} f = \mathbf{t}$.

Our proofs will use unique/affine existence via Proposition 3.1.3:

Proposition 3.1.3 *Suppose V is a set and $f : V \to \mathbf{3}$ is a function and $v, v' \in V$. Then:*

(1) *The following are equivalent:*
  (a) $\vDash \exists_{\mathbf{01}} f$.
  (b) $\exists_{\mathbf{01}} f \in \{\mathbf{t}, \mathbf{b}\}$.
  (c) $\exists_{01} v{\in}V.(\vDash \mathsf{T}\, f(v))$.
  (d) $\exists_{01} v{\in}V.f(v) = \mathbf{t}$.
  (e) $\forall v, v'{\in}V.f(v) = \mathbf{t} = f(v') \implies v = v'$.
(2) *The following are equivalent:*
  (a) $\vDash \exists_{\mathbf{1}} f$.
  (b) $\exists v{\in}V.(\vDash f(v)) \ \wedge\ \vDash \exists_{\mathbf{01}} f$.
(3) *The following are equivalent:*
  (a) $\vDash \mathsf{T}\, \exists_{\mathbf{01}} f$.
  (b) $\exists_{01} v{\in}V.f(v) \in \{\mathbf{t}, \mathbf{b}\}$.
(4) *The following are equivalent:*
  (a) $\vDash \mathsf{T}\, \exists_{\mathbf{1}} f$.
  (b) $\exists_1 v{\in}V.f(v) = \mathbf{t}$ *and* $\forall v{\in}V.f(v) \neq \mathbf{b}$.
(5) *If $\vDash \exists_{\mathbf{01}} f$ or $\vDash \exists_{\mathbf{1}} f$, and $\vDash \mathsf{T}\,(f(v) \wedge f(v'))$, then $v = v'$.*

Proof. We unpack Definition 3.1.1 and check the possibilities in Remark 3.1.2. □

3.2 A simple modal logic

Definition 3.2.1 We fix an infinite set of **variable symbols** $a, b \in \mathsf{VarSymb}$. We assume a nonempty set of **values** $v \in \mathsf{Val}$ and a set of **predicate symbols** $\mathsf{P} \in \mathsf{PredSymb}$. Specifically:

(1) For our example of Bracha Broadcast in Definition 4.1.2, we set Val to be any nonempty set and we take $\mathsf{PredSymb} = \{\mathsf{bcst}, \mathsf{echo}, \mathsf{ready}, \mathsf{dlvr}\}$.

$$
\begin{array}{lll}
\text{Terms} & t ::= a \mid v \quad (a \in \mathsf{VarSymb},\ v \in \mathsf{Val}) & \\
\text{Predicates} & \phi ::= \bot \mid \neg\phi \mid \phi \wedge \phi \mid t{=}t \mid \Box\,\phi \mid \boxdot\phi & \\
 & \quad \mid \exists a.\phi \mid \exists_{\mathbf{01}} a.\phi \mid \mathsf{TF}\,\phi \mid \mathsf{P}(t) \quad (\mathsf{P} \in \mathsf{PredSymb}) & \\
\text{Sugar} & \Diamond\phi = \neg\Box\neg\phi \qquad \diamondsuit\phi = \neg\boxdot\neg\phi & \phi \vee \phi' = \neg(\neg\phi \wedge \neg\phi') \\
 & \mathsf{B}\,\phi = \neg\mathsf{TF}\,\phi \qquad \mathsf{T}\,\phi = \phi \wedge \mathsf{TF}\,\phi & \phi \oplus \phi' = (\phi \wedge \neg\phi') \vee (\neg\phi \wedge \phi') \\
 & \phi \to \phi' = \neg\phi \vee \phi' \quad \phi \rightharpoonup \phi' = \phi \to \mathsf{T}\,\phi' & \\
 & \forall a.\phi = \neg\exists a.\neg\phi \quad \exists_{\mathbf{1}} a.\phi = \exists_{\mathbf{01}} a.\phi \wedge \exists a.\phi & \\
 & \mathsf{TF}[\mathsf{P}_1,\ldots,\mathsf{P}_n] = \forall a.(\mathsf{TF}\,\mathsf{P}_1(a) \wedge \cdots \wedge \mathsf{TF}\,\mathsf{P}_n(a)) \qquad (\mathsf{P}_1,\ldots,\mathsf{P}_n \in \mathsf{PredSymb}) & \\
 & \mathsf{B}[\mathsf{P}_1,\ldots,\mathsf{P}_n] = \forall a.(\mathsf{B}\,\mathsf{P}_1(a) \wedge \cdots \wedge \mathsf{B}\,\mathsf{P}_n(a)) &
\end{array}
$$

Fig. 7. Syntax of a simple modal logic (Definition 3.2.1)

(2) For our example of Crusader Agreement in Definition 5.1.2, we set $\mathsf{Val} = \{0, \frac{1}{2}, 1\}$ and we take $\mathsf{PredSymb} = \{\mathsf{input}, \mathsf{echo}_1, \mathsf{echo}_2, \mathsf{output}\}$.

We then define **terms** t, **predicates** ϕ, and syntactic sugar as in Figure 7.

Write $\phi[a{:=}v]$ for **substitution** of v for a; this is the result of replacing every unbound a in ϕ with v.

Notation 3.2.2 We may abuse notation in two ways:

(1) We may write v for quantified variables, e.g. writing $\exists v.\mathsf{echo}(v)$ instead of $\exists a.\mathsf{echo}(a)$. We may also write an (implicitly) universally quantified variable as v instead of as a; e.g. we do this in Figures 11 and 14.

The reader is welcome to mentally rewrite v back to a, or to decree that the v in Figures 11 and 14 is actually a variable symbol. [23] Meaning should be quite clear.

(2) We may also elide quantified variables entirely, in the style of higher-order logic. E.g. we may write $\exists\mathsf{echo}$ for $\exists a.\mathsf{echo}(a)$, or $\exists_{\mathbf{01}}\diamondsuit\mathsf{dlvr}$ for $\exists_{\mathbf{01}} a.\diamondsuit\mathsf{dlvr}(a)$. We will only do this where it is unambiguous how to fill in the relevant variable symbol if required.

Definition 3.2.3

(1) Given a semitopology $(\mathsf{Pnt}, \mathsf{Opn})$, an **interpretation** $\varsigma : \mathsf{PredSymb} \to \mathsf{Pnt} \to \mathsf{Val} \to \mathbf{3}$ is an assignment to each $\mathsf{P} \in \mathsf{PredSymb}$ of a function $\varsigma(\mathsf{P}) : \mathsf{Pnt} \to \mathsf{Val} \to \mathbf{3}$.

(2) A **model** $\mu = (\mathsf{Val}, (\mathsf{Pnt}, \mathsf{Opn}), \varsigma)$ consists of a set of values, a semitopology, and an interpretation. [24]

(3) Given a model μ, and a closed predicate ϕ (no free variable symbols; it may contain values), and $p \in \mathsf{Pnt}$, we define notions of
- **denotation** $[\![\phi]\!]_\mu : \mathsf{Pnt} \to \mathbf{3}$ and
- **validity** $p \models_\mu \phi$ and $\models_\mu \phi$

as per Figure 8.

(4) An **axiom** is a closed predicate ϕ. [25] Call an axiom ϕ **valid** in μ when $\models_\mu \phi$. Call a set of axioms THY a **theory**.

(5) Write $\models_\mu \textsc{Thy}$ when $\forall\phi{\in}\textsc{Thy}.(\models_\mu \phi)$ (i.e. the axioms of THY are all valid in μ), in which case we call

[23] This kind of overloading between object-level variable symbols and discourse-level variable symbols is not uncommon in mathematical discourse; e.g. if asked what 'x' is in $\forall x \in \mathbb{N}.x = x$ the answer 'it's a number' would be considered just as valid as 'it's a variable symbol denoting a number'. For extra pedant-points, note that a and b are arguably not actually variable symbols; they are arguably discourse-level variables ranging over distinct object-level variable symbols.
As my set-theory teacher once advised me when I asked if x in the course was a set, a variable denoting a set, or a variable denoting a variable denoting a set: don't think too hard about this, for that way lies madness.

[24] In practice we might wish to include $\mathsf{PredSymb}$ explicitly in μ, but note that this is mathematically redundant, in the sense that $\mathsf{PredSymb}$ can be recovered as the domain of ς.

[25] The terminology reflects intent: if we point at a closed predicate ϕ and say 'This ϕ is an axiom!', it carries a connotation that we may intend to restrict to models in which ϕ is *valid*.

$$\begin{aligned}
[\![\bot]\!]_\mu(p) &= \mathbf{f} & [\![\neg\phi]\!]_\mu(p) &= \neg[\![\phi]\!]_\mu(p)\\
[\![\phi \wedge \phi']\!]_\mu(p) &= [\![\phi]\!]_\mu(p) \wedge [\![\phi']\!]_\mu(p)\\
[\![\Box\,\phi]\!]_\mu(p) &= \Box\,[\![\phi]\!]_\mu & [\![\boxdot\phi]\!]_\mu(p) &= \boxdot[\![\phi]\!]_\mu\\
[\![\exists a.\phi]\!]_\mu(p) &= \exists\lambda v.[\![\phi[a{:=}v]]\!]_\mu(p) & [\![\exists_{\mathbf{01}} a.\phi]\!]_\mu(p) &= \exists_{\mathbf{01}}\lambda v.[\![\phi[a{:=}v]]\!]_\mu(p)\\
[\![\mathsf{TF}\,\phi]\!]_\mu(p) &= \mathsf{TF}\,([\![\phi]\!]_\mu(p))\\
[\![\mathsf{P}(v)]\!]_\mu(p) &= \varsigma(\mathsf{P})(p)(v) \quad (\mathsf{P} \in \mathsf{PredSymb})\\
[\![v{=}v']\!]_\mu(p) &= \begin{cases}\mathbf{t} & v = v'\\ \mathbf{f} & v \neq v'\end{cases}
\end{aligned}$$

$$\begin{aligned}
p \models_\mu \phi \;\text{ means }\; [\![\phi]\!]_\mu(p) \in \{\mathbf{t},\mathbf{b}\} \qquad \models_\mu \phi \;\text{ means }\; \forall p{\in}\mathsf{Pnt}.(p \models_\mu \phi)\\
\models_\mu \Phi \;\text{ means }\; \forall\phi{\in}\Phi.(\models_\mu \phi) \qquad \Phi \models_\mu \phi \;\text{ means }\; \models_\mu \Phi \implies \models_\mu \phi
\end{aligned}$$

Above: ϕ is a closed predicate (no free variables; it may contain values). $\exists$ and $\exists_{\mathbf{01}}$ to the left of $=$ are syntax from Figure 7, and to the right of $=$ they are functions as per Figure 6. Similarly $\Box$ and $\boxdot$ to the left of $=$ are syntax, and to the right of $=$ they are functions as per Definition 2.3.2. The $\mathsf{=}$ in $v{=}v$ on the left is the symbol in our predicate syntax from Figure 7; the other 'equals' symbols are normal mathematical equality.

Fig. 8. Denotation for a simple modal logic (Definition 3.2.3)

μ a **model of** Thy.

(6) Write $\text{Thy} \models_\mu \phi$ when $\models_\mu \text{Thy}$ implies $\models_\mu \phi$ (i.e. if μ is a model of Thy, then ϕ is valid in μ).

Notation 3.2.4 In what follows, we may use the fact that $p \models_\mu \Diamond\phi$ is equivalent to $\models_\mu \Diamond\phi$ without comment, and similarly for the other modalities $\Box$, $\boxdot$, and $\diamondsuit$, and for compound predicates like $\Diamond\phi \to \Box\phi$. This pattern will be quite common in the proofs that follow, because many of our axioms have the form $\phi \to \boxdot\phi'$, so when we use weak modus ponens (Prop. 2.2.3(4)) we may assume $p \models_\mu \mathsf{T}\,\phi$ and deduce $\models_\mu \boxdot\phi'$ (not $p \models_\mu \boxdot\phi'$); the p is still there in some sense, but it no longer matters.

Remark 3.2.5 We should check that the choice of sugared symbols in Figure 7 matches up with what we would expect from the truth-tables in Figure 1 and the denotations in Figure 8.

Thus (for example) in Figure 8 we fix the denotations $[\![\neg\phi]\!]$ and $[\![\phi \wedge \phi']\!]$, and in Figure 7 we define $\phi \vee \phi'$ as sugar for $\neg(\neg\phi \wedge \neg\phi')$ and $\phi \oplus \phi'$ as sugar for $(\phi \wedge \neg\phi') \vee (\neg\phi \wedge \phi')$ and $\phi \to \phi'$ as sugar for $\neg\phi \vee \phi'$ and $\mathsf{T}\,\phi'$ as sugar for $\phi' \wedge \mathsf{TF}\,\phi'$ and $\phi \rightharpoonup \phi'$ as sugar for $\phi \to \mathsf{T}\,\phi'$.

So if we unpack [-] on $\vee$, $\oplus$, $\to$, T, and $\rightharpoonup$ as *sugar*, do we arrive at the truth-tables for $\vee$, $\oplus$, $\to$, T, and $\rightharpoonup$ respectively in Figure 1?

We do. Checking this is completely straightforward.

4 Declarative Bracha Broadcast

4.1 Definition of the theory

Remark 4.1.1 Bracha Broadcast is a classic broadcast algorithm [7]. An English description of the algorithm is given in Figure 9. It is designed to allow a group of participants to agree on at most one value, in the presence of some hostile participants.

Definition 4.1.2 Define **Declarative Bracha Broadcast** to be the theory (set of axioms) ThyBB in Figure 10. Axioms are taken to be universally quantified over any free variables (there is only one: a). As per Definition 3.2.1(1), we set Val to be any nonempty set and $\mathsf{PredSymb} = \{\mathsf{bcst}, \mathsf{echo}, \mathsf{ready}, \mathsf{dlvr}\}$.

Remark 4.1.3 A fundamental difference between a logical theory and an algorithm is:

- With a theory, we are presented with a model and we ask "Are the axioms valid in this model?".

Protocol description.

(1) A designated *sender* participant sends messages to all participants, **broadcasting** a value v. Message-passing is reliable, so messages always arrive.
(2) If a participant receives a broadcast v message from the sender – it is assumed that every participant knows who the sender is, and other participants cannot forge the sender's signature – it sends an **echo** v to all other participants.
    Each participant will only echo *once*, so if it receives two broadcast messages with different values then it will only echo one of them.
(3) If a participant receives a quorum of echo messages for a value v, or a contraquorum of ready messages for a value v, then it sends messages to all participants declaring itself **ready** with v.
(4) If a participant receives a quorum of ready messages for a value v, then the participant **delivers** v.

Failure assumptions. Our failure assumption will be that a quorum of participants follows the algorithm, and a coquorum (the complement of a quorum, i.e. a nontotal closed set) of participants may not follow the algorithm (more on this terminology in Remark 2.3.4). For instance, a hostile participant may broadcast or echo multiple values v.

Fig. 9. Informal (English) description of Bracha Broadcast (Remark 4.1.1)

Backward rules		*Forward rules*	
(BrDeliver?)	$\mathsf{dlvr}(a) \to \boxdot\mathsf{ready}(a)$	(BrDeliver!)	$\boxdot\mathsf{ready}(a) \to \mathsf{dlvr}(a)$
(BrReady?)	$\mathsf{ready}(a) \to \boxdot\mathsf{echo}(a)$	(BrReady!)	$\boxdot\mathsf{echo}(a) \to \mathsf{ready}(a)$
(BrEcho?)	$\mathsf{echo}(a) \to \Diamond\,\mathsf{bcst}(a)$	(BrEcho!)	$\Diamond\,\mathsf{bcst}(a) \to \exists\mathsf{echo}$
		(BrReady!!)	$\diamondsuit\mathsf{ready}(a) \to \mathsf{ready}(a)$
Other rules			
(BrEcho01)	$\exists_{\mathbf{01}}\,\mathsf{echo}$	(BrCorrect)	$\boxdot\mathsf{TF}[\mathsf{ready}] \wedge \boxdot\mathsf{TF}[\mathsf{echo}]$
(BrBroadcast1)	$\exists_{\mathbf{1}}\Diamond\,\mathsf{bcst}$	(BrCorrect′)	$\mathsf{TF}[\mathsf{P}] \vee \mathsf{B}[\mathsf{P}]$ $(\mathsf{P}\in\{\mathsf{ready},\mathsf{echo}\})$
		(BrCorrect″)	$\Box\,\mathsf{TF}[\mathsf{bcst}] \vee \Box\,\mathsf{B}[\mathsf{bcst}]$

Free a above are assumed universally quantified (i.e. there is an invisible $\forall a$ at the start of any axiom with a free a). The axioms above assume predicate symbols $\mathsf{PredSymb} = \{\mathsf{bcst}, \mathsf{echo}, \mathsf{ready}, \mathsf{dlvr}\}$, as per Definition 3.2.1. We use Notation 3.2.2(2) to elide quantified variable symbols in (BrEcho01) and (BrBroadcast1).

Fig. 10. ThyBB: axioms of Declarative Bracha Broadcast (Definition 4.1.2)

(BrValidity)	ThyBB $\models_\mu \Diamond\,\mathsf{bcst}(v) \to \Box\,\mathsf{dlvr}(v)$	(Proposition 4.2.10)
(BrNoDup)	ThyBB $\models_\mu \exists_{\mathbf{01}}\mathsf{dlvr}$	(Proposition 4.2.12)
(BrIntegrity)	ThyBB $\models_\mu \mathsf{dlvr}(v) \to \Diamond\,\mathsf{bcst}(v)$	(Proposition 4.2.13)
(BrConsistency)	ThyBB $\models_\mu \exists_{\mathbf{01}}\Diamond\,\mathsf{dlvr}$	(Proposition 4.2.12)
(BrTotality)	ThyBB $\models_\mu \Diamond\,\mathsf{dlvr}(v) \to \Box\,\mathsf{dlvr}(v)$	(Proposition 4.2.14)

Above, we use Notation 3.2.2(2) to elide quantified variable symbols in (BrNoDup) and (BrConsistency).

Fig. 11. Correctness properties for ThyBB (Remark 4.2.2)

- With an algorithm, we start from nothing and we construct a model by following the algorithm.

So in this paper, we assume a model is just presented to us, and our business is to check whether it validates axioms. All of the complexity inherent in studying an implementation – i.e. a machine to compute such a model – is elided by design.

Remark 4.1.4 As for our voting example from Figure 3, the axioms in Figure 10 are of three kinds:

(1) *Backward rules* have names ending with ? and intuitively represent reasoning of the form "if *this* happens, then *that* must have happened".
The reader might find it helpful to think of these as *safety* or *correctness rules*, but 'backward rule' is more neutral and descriptive. Correctness is a theorem *about* a protocol. A protocol might fail to be correct and still be described using well-formed backward rules; they just specify a protocol for which the desired correctness property does not hold.

(2) *Forward rules* have names ending with ! and intuitively represent forward reasoning of the form "if *this* happens, then *that* must happen".
The reader might find it helpful to think of these as *liveness rules*, but 'forward rule' is more neutral and descriptive. A protocol can fail to be live, yet still be described using forward rules.

(3) *Other rules* reflect particular conditions of the algorithms, e.g. having to do with failure assumptions.

Remark 4.1.5 [Discussion of the axioms] We consider the axioms in turn. Each is implicitly universally quantified over a if required; so the a in (BrDeliver?) represents 'any value $v \in \mathsf{Val}$'. The truth-values $\mathbf{t}$ and $\mathbf{f}$ represent honest behaviour, and $\mathbf{b}$ represents dishonest behaviour. Each axiom is evaluated at an arbitrary $p \in \mathsf{Pnt}$:

(1) (BrDeliver?) says: if $\mathsf{dlvr}(v)$ is $\mathbf{t}$ at p then for a quorum of participants $\mathsf{ready}(v)$ is $\mathbf{t}$ or $\mathbf{b}$.
This reflects that in the algorithm as outlined in Remark 4.1.1, if p honestly delivers v then a quorum of participants must have sent ready messages for v, though some of those messages may have been sent by dishonest participants.
(2) (BrReady?) says: if $\mathsf{ready}(v)$ is $\mathbf{t}$ at p then for a quorum of participants $\mathsf{echo}(v)$ is $\mathbf{t}$ or $\mathbf{b}$.
(3) (BrEcho?) says: if $\mathsf{echo}(v)$ is $\mathbf{t}$, then somebody $\mathsf{bcst}(v)$ with $\mathbf{t}$ or $\mathbf{b}$.
(4) (BrDeliver!) says: if $\mathsf{ready}(v)$ is $\mathbf{t}$ for a quorum of participants, then $\mathsf{dlvr}(v)$ is $\mathbf{t}$ or $\mathbf{b}$.
This reflects that in the algorithm, if a quorum of honest participants send ready-v messages and if p is honest, then p will send a deliver-v message.
(5) (BrReady!) says: if $\mathsf{echo}(v)$ is $\mathbf{t}$ for a quorum of participants, then $\mathsf{ready}(v)$ is $\mathbf{t}$ or $\mathbf{b}$.
(6) (BrEcho!) says: if bcst is $\mathbf{t}$ for some participant and some value, then echo is $\mathbf{t}$ or $\mathbf{b}$ for some value, though not necessarily the same one.
This reflects that in the algorithm, if somebody honestly broadcasts some value and if p is honest, then p will echo some value. The fact that this will be the *same* value (if both parties are honest) is a Lemma that follows using (BrEcho?); see Lemma 4.2.7(1).
(7) (BrReady!!) says: if $\mathsf{ready}(v)$ is $\mathbf{t}$ for a *contraquorum* of participants, then $\mathsf{ready}(v)$ is $\mathbf{t}$ or $\mathbf{b}$. This reflects the disjunct 'or a contraquorum of ready messages' in Remark 4.1.1(3).
(8) (BrEcho01) says that $\mathsf{echo}(v)$ is $\mathbf{t}$ for at most one value v. It asserts nothing about byzantine behaviour: $\mathsf{echo}(v)$ can be $\mathbf{b}$ for as many values as we like, reflecting the fact that dishonest participants may deviate from the protocol.
(9) (BrBroadcast1) says that there is *precisely* one value v such that some participant $\mathbf{t}$-broadcasts v (so a model in which two participants $\mathbf{t}$-broadcast distinct values would *not* satisfy this condition, whereas a model in which all participants $\mathbf{t}$-broadcast the same value and $\mathbf{f}$-broadcast all other values, would) *or* there is *at least* one value v such that some participant $\mathbf{b}$-broadcasts v. This is what $\exists_{\mathbf{1}}$ means in a three-value setting, as discussed in Remark 3.1.2 and Proposition 3.1.3.
(10) (BrCorrect) asserts that there are quorums of honest participants for ready and echo. The literature tends to assume something slightly stronger, that there is one quorum of honest participants for all predicates. We will only need this weaker assumption for our proofs.
(11) (BrCorrect$'$) asserts that a participant p is either entirely honest for ready and echo, in that it returns $\mathbf{t}$ or $\mathbf{f}$ for every value, or it is entirely dishonest, in that it returns $\mathbf{b}$ for every value. This accurately reflects an informal assumption that (for a given predicate) a participant is either honest or dishonest – dishonesty is not attached to a particular message, but to the *participant* who sent it.
(BrCorrect$''$) asserts that either for all participants the bcst predicate returns $\mathbf{t}$ or $\mathbf{f}$ on every v, or for all participants it returns $\mathbf{b}$. This reflects that we do not designate a unique sender:

Remark 4.1.6 Implementations of Bracha Broadcast assume a unique *sender* participant whose identity is known to all participants and whose identity cannot be faked. The sender's task is to pick a unique

value v and broadcast v (and, if the sender is honest, only v) to all participants.

In our logic, we model this with a straightforward axiom $\exists_{\mathbf{1}} a.\Diamond \mathsf{bcst}(a)$. Our logic is three-valued, and as per Proposition 3.1.3(5) this means that

- there is at most one v such that there exists some participant that **t**-sends v, but
- there may be many values $v_1, \ldots$ such that for each v_i there exists some participant that **b**-sends v_i.

We return to this in Remark 4.2.5, after we have a few more proofs.

Remark 4.1.7 Formally, $\models_\mu$ ThyBB means that μ is a model and the axioms of ThyBB are valid of μ. Intuitively, $\models_\mu$ ThyBB is intended to mean "μ represents a run of Bracha Broadcast", such that:

- $p \models_\mu \mathsf{bcst}(v)$ means "p broadcast v at some point in the run".
- $p \models_\mu \mathsf{echo}(v)$ means "p echoed v at some point in the run".
- $p \models_\mu \mathsf{ready}(v)$ means "p declared itself ready for v at some point in the run".
- $p \models_\mu \mathsf{dlvr}(v)$ means "p delivered v at some point in the run".

Note however that these are just intuitions. The model is just a mathematical structure; $\mathsf{bcst}(v)$, $\mathsf{echo}(v)$, $\mathsf{ready}(v)$, and $\mathsf{dlvr}(v)$ are not messages, they are predicates. As per Remark 4.1.3 we do not assume anything about where μ came from and in particular, there is no *a priori* guarantee that μ was *actually generated* by a run of an actual implementation of the Bracha Broadcast algorithm. μ is just a structure of which we assume that $\models_\mu$ ThyBB. This abstraction is a feature, not a bug; μ is intended to be abstract.

Proving (or to use a more suggestive term: *formally verifying*) that a *particular* concrete implementation of Bracha Broadcast *actually does* lead to models that satisfy ThyBB, is the familiar problem of checking that an implementation satisfies its specification. Consider for example proving that a *particular* implementation of a smart contract *actually does* satisfy a token specification [15], or that a *particular* implementation of addition *actually does* add numbers.[26] So ThyBB is a way to declaratively specify what it is about an implementation that makes it be an implementation *of* Bracha Broadcast, and to the extent that we may agree that ThyBB does indeed capture this intuition in formal axiomatic form, ThyBB can claim to *be* 'Declarative Bracha Broadcast'.

4.2 Correctness properties for Bracha Broadcast

4.2.1 Statement of the correctness properties

Remark 4.2.1 For the rest of this section, we fix a model (Definition 3.2.3)

$$\mu = (\mathsf{Val}, (\mathsf{Pnt}, \mathsf{Opn}), \varsigma)$$

such that the underlying semitopology $(\mathsf{Pnt}, \mathsf{Opn})$ is 3-twined (Definition 2.4.1), so that we have Theorem 2.4.5.

Remark 4.2.2 As promised (see end of Remark 4.1.1), we will now use ThyBB in Figure 10 to derive predicates corresponding to the following standard correctness properties for Bracha Broadcast, per Figure 11. We consider them in turn:

(1) **Validity:** *If a correct p broadcasts a value v, then every correct p' delivers v.*
This becomes ThyBB $\models_\mu \Diamond \mathsf{bcst}(v) \to \Box\, \mathsf{dlvr}(v)$. See Proposition 4.2.10.

(2) **No duplication:** *Every correct p delivers at most one value.*
This becomes ThyBB $\models_\mu \exists_{\mathbf{01}} \mathsf{dlvr}$. See Proposition 4.2.12.

[26] Note that this can be more subtle than it sounds, even on an apparently innocuous example like addition on numbers. For example: is 65535+1 equal to 65536, or 0, or an overflow error? Perhaps 65535 is not a number at all (it is not if our notion of number is 8-bit integers). So the answer depends on what kind of 'addition' we mean, on what kind of 'number'. Behind the deceptively innocuous English phrase 'a *particular* implementation of addition' can lie many possible formal meanings; signed integer of various bitlengths, unsigned integers, floating point datatypes, arbitrary precision integers, finite field elements, real numbers, complex numbers, quaternions, ... At this point, we see that careful abstraction and logical specification are not an indulgence but a necessary default, and two specifications being different does not necessarily mean that one of them must be wrong; they may be capturing different formal entities that mere English may gloss over and/or struggle to faithfully express.

(3) **Integrity:** *If some p delivers a message m with correct sender p', then m was broadcast by p'.*
This becomes ThyBB $\models_\mu \mathsf{dlvr}(v) \to \Diamond\,\mathsf{bcst}(v)$. See Proposition 4.2.13.
(4) **Consistency:** *If correct p delivers v and another correct p' delivers v' then $v{=}v'$.*
This becomes ThyBB $\models_\mu \exists_{\mathbf{01}}\Diamond\,\mathsf{dlvr}$. See Proposition 4.2.12.
(5) **Totality:** *If correct p delivers v, then every correct p' delivers v.*
This becomes ThyBB $\models_\mu \Diamond\,\mathsf{dlvr}(v) \to \Box\,\mathsf{dlvr}(v)$. See Proposition 4.2.14.

The English statements above are (in order) adapted from properties BCB1, BCB2, BCB3, and BCB4 from Module 3.11, and BRB5 from Module 3.12 of [9].

Remark 4.2.3 We can make a few observations about the correctness properties above:

(1) The logical statements are concise, precise, and (arguably) clear.
(2) The rendering of Consistency $\exists_{\mathbf{01}}\Diamond\,\mathsf{dlvr}$ in (BrConsistency) subsumes that for No Duplication $\exists_{\mathbf{01}}\mathsf{dlvr}$ in (BrNoDup).
This is less visible in the English because Consistency talks about a 'correct p' and 'another correct p'', which might be taken to indicate that $p \neq p'$. In our logic, it is easier to unify Consistency and No Duplication into a single predicate and proof.
(3) The English statement of Integrity was hard to parse, at least for me. Contrast with the predicate $\mathsf{dlvr}(v) \to \Diamond\,\mathsf{bcst}(v)$, which is clear and precise.

4.2.2 Validity: "if a correct p broadcasts v, then every correct p' delivers v"

Lemma 4.2.4 *Suppose* $\models_\mu$ ThyBB. *Then precisely one of the following holds:*

(1) $(\forall p \in \mathsf{Pnt}.p \models_\mu \mathsf{TF}[\mathsf{bcst}]) \wedge \exists_1 v \in \mathsf{Val}.\forall p \in \mathsf{Pnt}.(p \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v))$, *or*
(2) $\forall v \in \mathsf{Val}, p \in \mathsf{Pnt}.(p \models_\mu \mathsf{B}\,\mathsf{bcst}(v))$.

Proof. (BrCorrect″) asserts that *either* for all participants the bcst predicate returns **t** or **f** on every v, *or* for all participants it returns **b**. In the latter case we have the latter (second) disjunct above, and in the former case using (BrBroadcast1) we obtain the former (first) disjunct.

Furthermore, these two disjuncts are mutually exclusive by non-emptiness of Pnt, as per Definition 2.3.1(1) (cf. Remark 2.3.8). □

Remark 4.2.5 Lemma 4.2.4 requires some discussion. Our description of Bracha Broadcast in Remark 4.1.1 assumes a designated sender with an unforgeable signature, who might be byzantine. This is an accurate description of the implementation, but logically we do not care that there is one sender. There could be *two* senders, so long as (if honest) they broadcast identical values.

Implementationally we guarantee this by designating a unique sender participant. In the axiomatisation, it is convenient (and more general) to be more abstract: we axiomatise a broadcast predicate that is either somewhere **t** for *precisely* one value (corresponding to a value broadcast by an honest sender), or it is **b** for all values and all participants (corresponding to arbitrary hostile behaviour of a dishonest sender). [27]

A technical fact will be helpful for Lemma 4.2.7:

Lemma 4.2.6 $\exists_1 v \in \mathsf{Val}.\forall p \in \mathsf{Pnt}.(p \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v))$ *implies* $\forall p \in \mathsf{Pnt}.\exists_1 v \in \mathsf{Val}.(p \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v))$.

Proof. It is simplest to consider the contrapositive: suppose the negation of the right-hand side.

- Suppose there exists $p \in \mathsf{Pnt}$ and two distinct values $v \neq v' \in \mathsf{Val}$ such that $p \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v)$ and $p \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v')$. But then it follows that $p' \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v)$ for *every* $p' \in \mathsf{Pnt}$, and also $p' \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v')$ for *every* $p' \in \mathsf{Pnt}$, which falsifies the left-hand side.
- Suppose there exists $p \in \mathsf{Pnt}$ such that for no $v \in \mathsf{Val}$ is it the case that $p \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v)$. But then it follows that for *every* $p' \in \mathsf{Pnt}$ it is not the case that $p' \models_\mu \mathsf{T}\Diamond\,\mathsf{bcst}(v)$, and the result follows. □

Lemma 4.2.7 *Suppose* $\models_\mu$ ThyBB *and* $v \in \mathsf{Val}$. *Then:*

[27] A longer stronger axiomatisation that is closer to the implementation could be written but would be more complex and not buy us much, since we can prove our correctness results from the weaker and simpler formulation.

(1) $\models_\mu \Diamond \mathsf{bcst}(v) \to \mathsf{echo}(v)$.
(2) $\models_\mu \Diamond \mathsf{bcst}(v) \to \Box\,\mathsf{echo}(v)$.
(3) $\models_\mu \boxdot\mathsf{echo}(v) \to \Box\,\mathsf{ready}(v)$.
(4) $\models_\mu \boxdot\mathsf{ready}(v) \to \Box\,\mathsf{dlvr}(v)$.

Proof. We consider each part in turn, freely using weak modus ponens (Prop. 2.2.3(4)) and the semantics in Figure 8:

(1) Consider $p \in \mathsf{Pnt}$ and suppose $p \models_\mu \mathsf{T}\Diamond\mathsf{bcst}(v)$. By weak modus ponens (Prop. 2.2.3(4)) and (BrEcho!) $p \models_\mu \exists a.\mathsf{echo}(a)$. Thus, there exists $v' \in \mathsf{Val}$ such that $p \models_\mu \mathsf{echo}(v')$. There are now two subcases:
- *Suppose* $p \models_\mu \mathsf{B}\,\mathsf{echo}(v')$. By (BrCorrect′) $p \models_\mu \mathsf{B}\,\mathsf{echo}(v)$, and so $p \models_\mu \mathsf{echo}(v)$.
- *Suppose* $p \models_\mu \mathsf{TF}\,\mathsf{echo}(v')$. By Proposition 2.2.3(8) (since $p \models_\mu \mathsf{echo}(v')$) $p \models_\mu \mathsf{T}\,\mathsf{echo}(v')$, and so by (BrEcho?) $p \models_\mu \Diamond\mathsf{bcst}(v')$. Now we assumed above that $p \models_\mu \mathsf{T}\Diamond\mathsf{bcst}(v)$ so this puts us in case 1 of Lemma 4.2.4. For clarity we write that case out in full:

$$\forall p \in \mathsf{Pnt}.p \models_\mu \mathsf{TF}[\mathsf{bcst}] \;\wedge\; \exists_1 v \in \mathsf{Val}.\forall p \in \mathsf{Pnt}.(p \models_\mu \mathsf{T}\Diamond\mathsf{bcst}(v)).$$

  From the left-hand conjunct and $p \models_\mu \Diamond\mathsf{bcst}(v')$ and Proposition 2.2.3(8) we have that $p \models_\mu \mathsf{T}\Diamond\mathsf{bcst}(v')$. But we assumed $p \models_\mu \mathsf{T}\Diamond\mathsf{bcst}(v)$ and so by the right-hand conjunct and Lemma 4.2.6 we have $v = v'$ as required.

(2) From part 1 of this result, noting that the $p \in \mathsf{Pnt}$ we chose in the proof was arbitrary.

(3) Suppose $p \models_\mu \mathsf{T}\boxdot\mathsf{echo}(v)$. By (BrReady!) $p \models_\mu \mathsf{ready}(v)$. This reasoning did not depend on p, so $\models_\mu \Box\,\mathsf{ready}(v)$ as required.

(4) As the reasoning for part 3, but using (BrDeliver!). □

Remark 4.2.8 For clarity we spell out what Lemma 4.2.7 asserts. Each part has the form $\models_\mu \phi \to \psi$, meaning that for every $p \in \mathsf{Pnt}$ if $p \models_\mu \mathsf{T}\,\phi$ (i.e. $[\![\phi]\!]_\mu(p) = \mathbf{t}$) then $p \models_\mu \psi$ (i.e. $[\![\psi]\!]_\mu(p) \in \{\mathbf{t}, \mathbf{b}\}$). So part 1 means '$p \models_\mu \mathsf{T}\Diamond\mathsf{bcst}(v)$ implies $p \models_\mu \mathsf{echo}(v)$'.

Note it does *not* mean '$p \models_\mu \Diamond\mathsf{bcst}(v)$ implies $p \models_\mu \mathsf{echo}(v)$'. That would be a different assertion.

Lemma 4.2.9 *Suppose* $\models_\mu$ ThyBB *and* $v \in \mathsf{Val}$. *Then:*

(1) $\models_\mu \Box\,\mathsf{echo}(v)$ *implies* $\models_\mu \mathsf{T}\boxdot\mathsf{echo}(v)$.
(2) $\models_\mu \Box\,\mathsf{ready}(v)$ *implies* $\models_\mu \mathsf{T}\boxdot\mathsf{ready}(v)$.

Proof. We consider each part in turn:

(1) Suppose $\models_\mu \Box\,\mathsf{echo}(v)$. By (BrCorrect) $\models_\mu \boxdot\mathsf{TF}[\mathsf{echo}]$ – by Figure 7 this means $\models_\mu \boxdot\forall v.\mathsf{TF}\,\mathsf{echo}(v)$ – thus $\models_\mu \boxdot\mathsf{TF}\,\mathsf{echo}(v)$ and by Lemma 2.3.6(2) and Proposition 2.2.3(8) $\models_\mu \mathsf{T}\boxdot\mathsf{echo}(v)$.

(2) As for part 1, since by (BrCorrect) $\models_\mu \boxdot\mathsf{TF}[\mathsf{ready}]$. □

Proposition 4.2.10 (Validity) *If* $\models_\mu$ ThyBB *and* $v \in \mathsf{Val}$ *then*

$$\models_\mu \Diamond\mathsf{bcst}(v) \to \Box\,\mathsf{dlvr}(v).$$

Proof. We reason using weak modus ponens (Prop. 2.2.3(4)). Suppose $\models_\mu \mathsf{T}\Diamond\mathsf{bcst}(v)$. By Lemma 4.2.7(2) $\models_\mu \Box\,\mathsf{echo}(v)$, and by Lemma 4.2.9(1) $\models_\mu \mathsf{T}\boxdot\mathsf{echo}(v)$. By Lemma 4.2.7(3) $\models_\mu \Box\,\mathsf{ready}(v)$, and by Lemma 4.2.9(2) $\models_\mu \mathsf{T}\boxdot\mathsf{ready}(v)$. By Lemma 4.2.7(4) $\models_\mu \Box\,\mathsf{dlvr}(v)$ as required. □

4.2.3 Consistency & No duplication: "if correct p delivers v and correct (possibly equal) p′ delivers v′ then v = v′"

Lemma 4.2.11 *Suppose* $\models_\mu$ (BrCorrect) *(in particular, it suffices that* $\models_\mu$ ThyBB*). Then for* $v, v' \in \mathsf{Val}$*:*

(1) *If* $\models_\mu \boxdot\mathsf{ready}(v)$ *then* $\models_\mu \mathsf{T}\diamondsuit\mathsf{ready}(v)$.
(2) *If* $\models_\mu \boxdot\mathsf{echo}(v) \wedge \boxdot\mathsf{echo}(v')$ *then* $\models_\mu \mathsf{T}\Diamond(\mathsf{echo}(v) \wedge \mathsf{echo}(v'))$.
(3) *If* $\models_\mu \boxdot\mathsf{ready}(v) \wedge \boxdot\mathsf{ready}(v')$ *then* $\models_\mu \mathsf{T}\Diamond(\mathsf{ready}(v) \wedge \mathsf{ready}(v'))$.

Proof. We consider each part in turn:

(1) Suppose $\models_\mu \boxdot\mathsf{ready}(v)$. From (BrCorrect) also $\models_\mu \boxdot\mathsf{T\!F}\,\mathsf{ready}(v)$. Using Theorem 2.4.5 $\models_\mu \diamondsuit\mathsf{T}\,\mathsf{ready}(v)$ and so $\models_\mu \mathsf{T}\diamondsuit\mathsf{ready}(v)$ as required.

(2) Suppose $\models_\mu \boxdot\mathsf{echo}(v) \wedge \boxdot\mathsf{echo}(v')$. From (BrCorrect) we have $\models_\mu \boxdot\mathsf{T\!F}[\mathsf{echo}]$. Using Theorem 2.4.5 and Lemma 2.3.7(3) (or just direct from the 3-twined property in Definition 2.4.1) $\models_\mu \mathsf{T}\diamondsuit(\mathsf{echo}(v) \wedge \mathsf{echo}(v'))$ as required.

(3) As for the proof of part 2, noting that $\models_\mu \boxdot\mathsf{T\!F}[\mathsf{ready}]$ from (BrCorrect). □

Proposition 4.2.12 (Consistency & No Duplication) *Suppose* $\models_\mu$ ThyBB. *Then*

$$\models_\mu \exists_{\mathbf{01}} v.\diamondsuit\,\mathsf{dlvr}(v).$$

Proof. By Proposition 3.1.3(1) it suffices to show for every $v, v' \in \mathsf{Val}$ that $\models_\mu \mathsf{T}\diamondsuit\,\mathsf{dlvr}(v) \wedge \mathsf{T}\diamondsuit\,\mathsf{dlvr}(v')$ implies $v = v'$. So suppose $\models_\mu \mathsf{T}\diamondsuit\,\mathsf{dlvr}(v)$ and $\models_\mu \mathsf{T}\diamondsuit\,\mathsf{dlvr}(v')$. By (BrDeliver?) $\models_\mu \boxdot\mathsf{ready}(v)$ and $\models_\mu \boxdot\mathsf{ready}(v')$, so by Lemma 4.2.11(3) $\models_\mu \mathsf{T}\diamondsuit(\mathsf{ready}(v) \wedge \mathsf{ready}(v'))$. By (BrReady?) $\models_\mu \boxdot\mathsf{echo}(v)$ and $\models_\mu \boxdot\mathsf{echo}(v')$, so by Lemma 4.2.11(2) $\models_\mu \mathsf{T}\diamondsuit(\mathsf{echo}(v) \wedge \mathsf{echo}(v'))$. It follows using (BrEcho01) and Proposition 3.1.3(5) that $v = v'$ as required. □

4.2.4 Integrity: "if some p delivers a message m with correct sender p′, then m was broadcast by p′"

Proposition 4.2.13 (Integrity) *Suppose* $\models_\mu$ ThyBB *and* $v \in \mathsf{Val}$. *Then*

$$\models_\mu \mathsf{dlvr}(v) \to \diamondsuit\,\mathsf{bcst}(v).$$

Proof. We reason using weak modus ponens (Prop. 2.2.3(4)). Suppose $p \in \mathsf{Pnt}$ and $p \models_\mu \mathsf{T}\,\mathsf{dlvr}(v)$. By (BrDeliver?) $\models_\mu \boxdot\mathsf{ready}(v)$. By Lemma 4.2.11(3) (taking $v = v'$ in that Lemma) $\models_\mu \mathsf{T}\diamondsuit\,\mathsf{ready}(v)$. By (BrReady?) $\models_\mu \boxdot\mathsf{echo}(v)$, and by Lemma 4.2.11(2) (taking $v = v'$) $\models_\mu \mathsf{T}\diamondsuit\,\mathsf{echo}(v)$. Using (BrEcho?) $\models_\mu \diamondsuit\,\mathsf{bcst}(v)$, as required. □

4.2.5 Totality: if correct p delivers v, then every correct p′ delivers v

Proposition 4.2.14 (Totality) *Suppose* $\models_\mu$ ThyBB *and* $v \in \mathsf{Val}$. *Then:*

$$\models_\mu \diamondsuit\,\mathsf{dlvr}(v) \;\to\; \Box\,\mathsf{dlvr}(v).$$

Proof. We reason using weak modus ponens (Prop. 2.2.3(4)). Suppose $\models_\mu \mathsf{T}\diamondsuit\,\mathsf{dlvr}(v)$. By (BrDeliver?) $\models_\mu \boxdot\mathsf{ready}(v)$, so by Lemma 4.2.11(1) $\models_\mu \mathsf{T}\diamondsuit\mathsf{ready}(v)$. Using (BrReady!!) (applied at every point) $\models_\mu \Box\,\mathsf{ready}(v)$, so by Lemma 4.2.9(2) $\models_\mu \mathsf{T}\boxdot\mathsf{ready}(v)$. Using (BrDeliver!) (applied at every point) $\models_\mu \Box\,\mathsf{dlvr}(v)$. □

Remark 4.2.15 Let us take a moment to unpack the statement of Totality in Proposition 4.2.14 and remember what the symbols mean. As per Figure 8 this asserts that $[\![\diamondsuit\,\mathsf{dlvr}(v) \to \Box\,\mathsf{dlvr}(v)]\!]_\mu(p)$ has truth-value **t** or **b** for every $p \in \mathsf{Pnt}$. The choice of p clearly does not matter here because both parts of the implication are modal. Unpacking the truth-table of $\to$ from Figure 1, it suffices to show that if the left-hand side has truth-value **t** then the right-hand side has truth-value **t** or **b**.

So suppose $[\![\diamondsuit\,\mathsf{dlvr}(v)]\!]_\mu = \mathbf{t}$, indicating that some honest participant delivers v. The proof of Proposition 4.2.14 then shows that: every other honest participant delivers v; for any dishonest participants $\mathsf{dlvr}(v)$ has truth-value **b**; and therefore, the truth-value of $\Box\,\mathsf{dlvr}(v)$ is **t** or (if dishonest participants are involved) **b**.

The statement and proof of Proposition 4.2.14 do not reason explicitly on correct or byzantine participants. They do not need to: this detail is all packaged neatly into how the modalities and implication work. The proof is routine, following the structure of the axioms and applying lemmas in a natural way.

It is easy to take this magic for granted: how logic turns reasoning into (almost) routine symbolic manipulation. But of course, this is why logical specifications can be so helpful. Declarative logical specifications of the kind we see above are particularly powerful, because of how concise they can be.

4.3 Further discussion of the axioms

We take a moment to return to the axioms in Figure 10, to discuss some design alternatives.

Axioms (BrDeliver!) and (BrReady!) in Figure 10 are slightly stronger than absolutely necessary. In the presence of the other axioms, it suffices to assume $\exists a.\boxdot\mathsf{ready}(a)\rightarrow\exists a.\mathsf{dlvr}(a)$ and $\exists a.\boxdot\mathsf{echo}(a)\rightarrow\exists a.\mathsf{ready}(a)$. Then it is not hard to use the backward rules to derive the stronger forms of the axioms as presented in Figure 10.

The same is not true of (BrReady!!). A weaker axiom $\exists a.\diamondsuit\mathsf{ready}(a) \rightarrow \exists a.\mathsf{ready}(a)$ would *not* suffice and the stronger form of the axiom cannot be derived from it.

Conversely, changing (BrEcho!) to $\diamondsuit\,\mathsf{bcst}(a)\rightarrow\mathsf{echo}(a)$ would be wrong. Consider the case of a byzantine participant who (dishonestly; contrary to the protocol) broadcasts v and v'. In our logic this would give $\diamondsuit\,\mathsf{bcst}(v)$ and $\diamondsuit\,\mathsf{bcst}(v')$ truth-value **b**. According to the truth-table for $\rightarrow$ in Figure 1, by our modified (BrEcho!) rule, $\mathsf{echo}(v)$ and $\mathsf{echo}(v')$ would then have to have truth-value **b** or **t**. For an honest participant this would mean that both $\mathsf{echo}(v)$ and $\mathsf{echo}(v')$ would have to have truth-value **t**. This would contradict (BrEcho01), which implies that no honest participant may echo more than one value.

Clause 3 of Remark 4.1.1 renders into logic as a forward rule $(\boxdot\mathsf{echo}(a)\,\mathsf{V}\,\diamondsuit\mathsf{ready}(a))\rightarrow\mathsf{ready}(a)$. This rule appears more-or-less verbatim in Figure 10 as (BrReady!) and (BrReady!!). Yet in Figure 10 we only have the backward rule (BrReady?) as $\mathsf{ready}(a)\rightarrow\boxdot\mathsf{echo}(a)$. Why not a rule (BrReady?′) of the form $\mathsf{ready}(a)\rightarrow(\boxdot\mathsf{echo}(a)\,\mathsf{V}\,\diamondsuit\mathsf{ready}(a))$? Algorithmically this is inefficient, because if somebody sends a ready message then by an inductive argument a quorum of echoes must exist. In the logic, (BrReady?′) would actually be *too weak*; it would permit a model in which every participant does $\mathsf{ready}(v)$, and for each p (BrReady?′) would be valid at p because of the contraquorum of all the *other* p' who also do $\mathsf{ready}(v)$. This is related to the observations in Remark 4.1.3: in the algorithmic world we build things, so things *can't* happen unless we say they *do*; in the logical world we verify things, so things *can* happen unless we say they *don't*.

5 Crusader Agreement

5.1 Motivation and presenting the protocol

Remark 5.1.1 In Section 4 we gave a declarative axiomatisation of the Bracha Broadcast protocol and we showed how to prove its correctness properties by axiomatic reasoning.

We will now do the same for a Crusader Agreement algorithm as presented in an online exposition [1]. This exposition was written for the Decentralized Thoughts website [28] by the authors of a journal paper [2], to provide an accessible introduction to their paper.

Agreement is arguably a moderate step up in complexity from Broadcast, because:

(1) With Broadcast, we want all correct participants to agree on a value broadcast by a single designated (possibly faulty) broadcasting participant.
(2) With Agreement, we want all correct participants to agree on some value, where all participants start off with their own individual values. While still simple, it is slightly more challenging in the sense that correct participants can propose different values whereas in Broadcast they cannot.

Definition 5.1.2

(1) Figure 12 presents the pseudocode from [1].
(2) Figure 13 presents **Declarative Crusader Agreement** to be the theory (set of axioms) ThyCA. Axioms are taken to be universally quantified over any free variables (there is only one: a). As per Definition 3.2.1(2), we set $\mathsf{Val} = \{0, \frac{1}{2}, 1\}$ and $\mathsf{PredSymb} = \{\mathsf{input}, \mathsf{echo}_1, \mathsf{echo}_2, \mathsf{output}\}$.

Remark 5.1.3 The authors of [1] are not completely explicit, but it becomes clear from context, that:

(1) Clauses in Figure 12 are intended to be executed in parallel (not necessarily sequentially).
(2) Each participant starts with their own input value, which may be different from the input value(s) of other participants.

[28] Decentralized Thoughts is a well-regarded source of technical commentary on decentralised protocols.

```
1  input: v (0 or 1)
2  send <echo1, v> to all
3  if didnt send <echo1, 1-v> and see f+1 <echo1, 1-v>
4      send <echo1, 1-v> to all
5  if didnt send <echo2, *> and see n-f <echo1, w>
6      send <echo2, w>
7  if see n-f <echo2, u> and n-f <echo1, u>
8      output(u)
9  if see n-f <echo1, 0> and n-f <echo1, 1>
10     output(0.5)
```

Fig. 12. Pseudocode Crusader Agreement algorithm from [1] (Definition 5.1.2)

Backward rules

(CaEcho1?)	$\mathsf{echo}_1(a) \rightharpoonup \Diamond\,\mathsf{input}(a)$
(CaEcho2?)	$\mathsf{echo}_2(a) \to \boxdot\mathsf{echo}_1(a)$
(CaOutput?)	$(\mathsf{output}(0) \to \boxdot\mathsf{echo}_2(0)) \wedge (\mathsf{output}(1) \to \boxdot\mathsf{echo}_2(1))$
(CaOutput′?)	$\mathsf{output}(\frac{1}{2}) \to (\boxdot\mathsf{echo}_1(0) \wedge \boxdot\mathsf{echo}_1(1))$

Forward rules

(CaEcho1!)	$(\mathsf{input}(a) \vee \dot\Diamond\mathsf{echo}_1(a)) \to \mathsf{echo}_1(a)$
(CaEcho2!)	$(\exists\boxdot\mathsf{echo}_1) \to \exists\mathsf{echo}_2$
(CaOutput!)	$\boxdot\mathsf{echo}_2(a) \to \mathsf{output}(a)$
(CaOutput′!)	$(\boxdot\mathsf{echo}_1(0) \wedge \boxdot\mathsf{echo}_1(1)) \to \mathsf{output}(\frac{1}{2})$

Other rules

(CaCorrect)	$\boxdot\mathsf{TF}[\mathsf{input}, \mathsf{echo}_1, \mathsf{echo}_2, \mathsf{output}]$
(CaCorrect′)	$\mathsf{TF}[\mathsf{P}] \vee \mathsf{B}[\mathsf{P}] \quad (\mathsf{P} \in \{\mathsf{input}, \mathsf{echo}_1, \mathsf{echo}_2, \mathsf{output}\})$
(CaInput)	$(\mathsf{input}(0) \oplus \mathsf{input}(1)) \wedge \neg\mathsf{input}(\frac{1}{2})$
($\mathsf{CaEcho2}_{01}$)	$\exists_{\mathbf{01}}\mathsf{echo}_2$

As standard, free a above are assumed universally quantified (i.e. there is an invisible $\forall a$ at the start of any axiom with a free a). The axioms above assume predicate symbols $\mathsf{PredSymb} = \{\mathsf{input}, \mathsf{echo}_1, \mathsf{echo}_2, \mathsf{output}\}$ (as explained in Definition 3.2.1). We use Notation 3.2.2(2) to elide quantified variable symbols in ($\mathsf{CaEcho2}_{01}$) and (CaEcho2!).

Fig. 13. ThyCA: axioms of Declarative Crusader Agreement (Definition 5.1.2)

Intuitively, in this algorithm participants try to agree on a value 0 or 1, and if a participant can see no clear choice of value to agree on, then it might output a special 'agreement failed' value. Here we write this value as $\frac{1}{2}$; in [1] they write $\perp$ (a riff on the domain-theoretic 'undefined' element), but that clashes with our use in this paper of $\perp$ for logical falsity in Figure 7.

Definition 5.1.4 Relevant correctness properties for Crusader Agreement as per [1] are presented in Figure 14. We consider them in turn:

(1) **Weak Agreement:** *If two non-faulty parties output values v and v', then either $v = v'$ or at least one of the values is $\frac{1}{2}$.*

(CaAgree)	$\textsc{ThyCA} \models_\mu (\Diamond \mathsf{output}(v) \wedge \Diamond \mathsf{output}(v')) \rightharpoonup (v{=}v' \vee v{=}\frac{1}{2} \vee v'{=}\frac{1}{2})$	(Proposition 5.3.3)
(CaValid1)	$\textsc{ThyCA} \models_\mu (\mathsf{TB}\,\Box\,\mathsf{input}(v) \wedge \mathsf{output}(v')) \rightharpoonup v{=}v'$	(Proposition 5.3.6)
(CaValid2)	$\textsc{ThyCA} \models_\mu \Diamond \mathsf{output}(v) \rightharpoonup (\Diamond\,\mathsf{input}(v) \vee v{=}\frac{1}{2})$	(Proposition 5.3.6)
(CaLive)	$\textsc{ThyCA} \models_\mu \Box\,\exists \mathsf{output}$	(Proposition 5.3.11)

We use Notation 3.2.2(2) to elide the quantified variable symbol in (CaLive).

Fig. 14. Correctness properties for ThyCA (Definition 5.1.4)

This becomes

$$\textsc{ThyCA} \models_\mu (\Diamond \mathsf{output}(v) \wedge \Diamond \mathsf{output}(v')) \rightharpoonup (v{=}v' \vee v{=}\tfrac{1}{2} \vee v'{=}\tfrac{1}{2}).$$

See Proposition 5.3.3.

(2) **Validity:** *If all non-faulty parties have the same input, then this is the only possible output* of any non-faulty party.[29] *Furthermore, if a non-faulty party outputs $v \neq \frac{1}{2}$, then v was the input of some non-faulty party.*
These become

$$\textsc{ThyCA} \models_\mu (\mathsf{TB}\,\Box\,\mathsf{input}(v) \wedge \mathsf{output}(v')) \rightharpoonup v{=}v' \quad \text{and}$$
$$\textsc{ThyCA} \models_\mu \Diamond \mathsf{output}(v) \rightharpoonup (\Diamond\,\mathsf{input}(v) \vee v{=}\tfrac{1}{2}).$$

See Proposition 5.3.6.

(3) **Liveness:** *If all non-faulty parties start the protocol then all non-faulty parties eventually output a value.*
This becomes $\textsc{ThyCA} \models_\mu \Box\,\exists v.\mathsf{output}(v)$. See Proposition 5.3.11.

5.2 Discussion of the axioms

Remark 5.2.1 As for the axioms in Figures 3 and 10, the axioms in Figure 13 are grouped into *backward rules* with names ending in ? which represent reasoning of the form "if *this* happens, then *that* must have happened"; *forward rules* have names ending with ! which represent forward reasoning of the form "if *this* happens, then *that* must happen"; and *other rules* which reflect particular conditions of the algorithms.

Recall that each axiom is implicitly universally quantified over a if required, and it is valid of a model when it is valid (i.e. it returns $\mathbf{t}$ or $\mathbf{b}$, but not $\mathbf{f}$) when evaluated at every $p \in \mathsf{Pnt}$.

Remark 5.2.2 We discuss each axiom in Figure 13 in turn:

(1) (CaEcho1?) says: if $\mathsf{echo}_1(v)$ is $\mathbf{t}$ at p then $\mathsf{input}(v)$ is $\mathbf{t}$ at some p'.
This axiom does not immediately correspond to lines 2 to 4 in Figure 12 — contrast with (CaEcho1!), which more clearly corresponds to the algorithm. We explain and discuss this difference below in Remark 5.2.3.

(2) (CaEcho2?) says: if $\mathsf{echo}_2(v)$ is $\mathbf{t}$ at p then for a quorum of participants $\mathsf{echo}_1(v)$ is $\mathbf{t}$ or $\mathbf{b}$. This corresponds to part of lines 5 and 6, since it is clear that if a participant performs $\mathsf{echo}_2(v)$ then it must have seen a quorum of $\mathsf{echo}_1(v)$.
The condition `if didnt send <echo2, *>` on line 5 is captured by ($\mathsf{CaEcho2}_{01}$), which just asserts that no correct participant may `echo2` *twice*.

(3) (CaOutput?) and (CaOutput!) reflect lines 7 to 10, except for one detail: lines 7 to 10 suggest a pair of axioms

$$\mathsf{output}(a) \rightarrow (\boxdot\mathsf{echo}_2(a) \wedge \boxdot\mathsf{echo}_1(a)) \qquad (\boxdot\mathsf{echo}_2(a) \wedge \boxdot\mathsf{echo}_1(a)) \rightarrow \mathsf{output}(a)$$

[29] I added the 'for any non-faulty party' (the original source material [1] did not say this) because from context it is clear that this is what is intended. Faulty parties are not constrained.

but instead in Figure 13 we have axioms

$$\mathsf{output}(a) \to \boxdot\mathsf{echo}_2(a) \qquad \boxdot\mathsf{echo}_2(a) \to \mathsf{output}(a).$$

Why the difference?

Because, in fact, the right-hand conjunct in the algorithm from [1] is redundant.[30] We could include it in the axioms and it would do no harm, but it also makes no difference to the proofs and would take up space.

One of the algorithm's authors[31] confirms this: the right-hand conjunct was left in by accident. It gives stronger correctness properties that are treated in the paper [2], but they are not mentioned in the (simplified) online article [1].

(4) ($\mathsf{CaOutput}'?$) asserts that if we output $\frac{1}{2}$ then we must have seen quorums of echo_1 for both 0 and 1, as per lines 9 and 10 in Figure 12.

(5) ($\mathsf{CaCorrect}$) asserts that there exists a quorum of correct participants, by which we mean precisely that they all return **t** or **f** truth-values for input, echo_1, echo_2, and output, and they never return **b**.

(6) ($\mathsf{CaCorrect}'$) asserts that *either* a participant is correct and always returns correct truth-values — *or* it always returns the faulty truth-value **b**.

The reader might ask why a faulty participant must always return **b**; what if it returns **b** just sometimes? The reason is that this logically captures worst-case behaviour. Specifically, the forward and backward rules are all implications, and it is a fact that if the left-hand side of an implication is **b**, then the implication overall returns **b** or **t** and so is valid; the reader can check this in Figure 1, just by observing that the row for **b** in the truth-tables for $\to$ and $\rightharpoonup$ is '**t b b**' and this does not mention **f**.

So, this makes **b** behave like a 'worst case truth-value' in the sense that all forward and backward rules are valid if the left-hand side returns **b**, so that in effect the rule does not constrain behaviour involving faulty participants. But, we do not need to keep saying 'for a non-faulty participant', because the three-valued logic takes care of all this for us, automatically.

(7) ($\mathsf{CaInput}$) expresses line 1: a correct participant inputs either 0 or 1.

(8) ($\mathsf{CaEcho1}!$) expresses the forward version of lines 2 to 4; echo1 the value of your input and echo1 the value of any contraquorum that you see.

(9) ($\mathsf{CaEcho2}!$) expresses the forward version of lines 5 and 6. We need the existential quantifier because in this case, we only echo2 the value of the first quorum of echo1 that we see.

(10) ($\mathsf{CaOutput}!$) was discussed above. We would just add here that $\mathsf{T}\,\mathsf{echo}_2(\frac{1}{2})$ is impossible by Lemma 5.4.2(3), so that in the presence of the other axioms, we do not need an explicit side-condition in ($\mathsf{CaOutput}!$) that $a \neq \frac{1}{2}$.

(11) ($\mathsf{CaOutput}'!$) corresponds directly to lines 9 and 10 in Figure 12 (and is just the reverse of ($\mathsf{CaOutput}'?$)).

Remark 5.2.3 ($\mathsf{CaEcho1}?$) is rather special. A direct reading of lines 2 to 4 of the algorithm from Definition 5.1.2 suggests we write the following property:

$$\mathsf{echo}_1(v) \to (\mathsf{input}(v) \mathsf{V} \diamondsuit\mathsf{echo}_1(v)).$$

Indeed, the forward rule ($\mathsf{CaEcho1}!$) looks just like that, just in the other direction. So why did we not write the predicate above as our ($\mathsf{CaEcho1}?$)?

Because it is subtly too weak. The axiom above would permit a model in which (for example) all participants are correct, and they all input 0, but they all perform $\mathsf{echo}_1(1)$. Note that $\mathsf{echo}_1(1) \to \diamondsuit\mathsf{echo}_1(1)$ is valid of this model. Clearly we want to exclude it, and no practical run of the algorithm would generate it, but logically speaking it perfectly well satisfies the restriction above.

What is going on here is that our model has no notion of algorithmic time; there is no notion of state transition system or abstract machine (see the discussion in Subsection 6.5). So, our model has no notion of a 'first' echo_1 that would need to be justified by an input.

[30] This became clear while doing the axiomatic proofs; the right-hand conjunct was never used. In fact the same is true in the source article, but this is harder to spot because its reasoning is less explicitly axiomatic.

[31] Ittai Abraham; private correspondence.

We could now be literal and introduce a notion of time, either as an explicit timestamp parameter to each echo_1, or we could add a notion of time to the modal context in the logic itself. There would be nothing wrong with that — it reflects what the algorithm actually does and is technically entirely feasible.

But we will prefer a simpler, more general, and arguably more elegant approach.[32] To explain how this works, we reason as follows: we observe of the algorithm that if a correct participant performs $\mathsf{echo}_1(v)$ then either *it* has input v, or $f{+}1$ participants — and thus in particular one correct participant — also have performed $\mathsf{echo}_1(v)$. By an induction on time, we see that if a correct participant performs $\mathsf{echo}_1(v)$ then some correct participant has input v. This justifies writing ($\mathsf{CaEcho1?}$) as we do in Figure 13.

It is not immediately obvious that this suffices to prove our correctness properties; we have to check that in the proofs. And, it turns out that the proofs work well, which gives a formal sense in which the ($\mathsf{CaEcho1?}$) axiom in Figure 13 is correct and in fact, it is more general than the more literal rendering.[33]

5.3 Proofs of correctness properties for Crusader Agreement

Remark 5.3.1 As we did for Bracha Broadcast in Remark 4.2.1, for the rest of this section, we fix a model (Definition 3.2.3) $\mu = (\mathsf{Val}, (\mathsf{Pnt}, \mathsf{Opn}), \varsigma)$ such that the underlying semitopology $(\mathsf{Pnt}, \mathsf{Opn})$ is 3-twined (Definition 2.4.1), so that we have Theorem 2.4.5 and Corollary 2.4.6.

5.3.1 Weak Agreement

Remark 5.3.2 The proof of Proposition 5.3.3 is straightforward. Note how the 'non-faulty' precondition is integrated into the use of $\rightharpoonup$, which assumes that the left-hand side returns $\mathbf{t}$ (the non-faulty truth-value).[34]

Proposition 5.3.3 (Weak Agreement) *Suppose* $\models_\mu$ ThyCA *and* $v, v' \in \{0, \frac{1}{2}, 1\}$. *Then*

$$\models_\mu (\Diamond \mathsf{output}(v) \wedge \Diamond \mathsf{output}(v')) \rightharpoonup (v{=}v' \vee v{=}\tfrac{1}{2} \vee v'{=}\tfrac{1}{2}).$$

This formalises the following informal statement:

"If two non-faulty parties output values v and v', then either $v = v'$ or one of the values is $\frac{1}{2}$"

Proof. Using strong modus ponens (Prop. 2.2.3(5)) it suffices to assume $\models_\mu \mathsf{T} \Diamond \mathsf{output}(v) \wedge \mathsf{T} \Diamond \mathsf{output}(v')$ and $v, v' \neq \frac{1}{2}$, and prove $v = v'$.

So suppose $\models_\mu \mathsf{T} \Diamond \mathsf{output}(v) \wedge \mathsf{T} \Diamond \mathsf{output}(v')$ and $v, v' \neq \frac{1}{2}$. By ($\mathsf{CaOutput?}$) $\models_\mu \boxdot \mathsf{echo}_2(v) \wedge \boxdot \mathsf{echo}_2(v')$. By ($\mathsf{CaCorrect}$) $\models_\mu \boxdot \mathsf{TF}[\mathsf{echo}_2]$. By Theorem 2.4.5 and Lemma 2.3.7(3) $\models_\mu \mathsf{T} \Diamond (\mathsf{echo}_2(v) \wedge \mathsf{echo}_2(v'))$. By ($\mathsf{CaEcho2}_{01}$) and Proposition 3.1.3(1.a&1.e) $v = v'$ as required. □

Example 5.3.4 output is not functional, by which we mean that $[\![\exists_{\mathbf{01}} v.\mathsf{output}(v)]\!]_\mu$ need not be equal to $\mathbf{t}$. We give an example to show that $p \models_\mu \mathsf{T}(\mathsf{output}(\frac{1}{2}) \wedge \mathsf{output}(1))$ is possible.

Consider a model with ten participants; all participants are correct; five participants input 0 and five input 1; quorums are any set with at least seven participants and thus contraquorums are any set with at least four participants. The reader can check that every participant necessarily produces $\mathsf{echo}_1(0)$ and $\mathsf{echo}_1(1)$, so that all participants $\mathsf{output}(\frac{1}{2})$. It is also consistent with the axioms for every participant to produce $\mathsf{echo}_2(1)$ and $\mathsf{output}(1)$.

5.3.2 Validity

We work towards Proposition 5.3.6; we will need Lemma 5.3.5:

Lemma 5.3.5 *Suppose* $\models_\mu$ ThyCA *and* $p \in \mathsf{Pnt}$. *Then:*

[32] But as always: what you axiomatise depends on what you want to represent. Today, we might make one design decision about how much detail we want to include in our model. Tomorrow, or for a different protocol, or for a different intended audience, it would be perfectly legitimate to choose different tradeoffs.

[33] Why is a weaker axiom better? Surely strong axioms with lots of structure are good? No: generally speaking, the game is to derive strong and well-structured correctness properties from weak and minimally-structured axioms; our game is to derive the strongest possible theorems from the weakest possible assumptions.

[34] On the right-hand side, $[\![v{=}v']\!] \in \{\mathbf{t}, \mathbf{f}\}$, so equality always returns a correct truth-value.

(1) $p \models_\mu \neg\mathsf{input}(\frac{1}{2})$.[35]
(2) $p \models_\mu \mathsf{input}(v)$ *if and only if* $p \models_\mu \neg\mathsf{input}(1\text{-}v)$, *for* $v \in \{0,1\}$.
(3) $p \models_\mu \mathsf{T}\,\mathsf{input}(v)$ *and* $p \models_\mu \mathsf{T}\,\mathsf{input}(v')$ *implies* $v = v'$, *for* $v, v' \in \{0, \frac{1}{2}, 1\}$.
(4) $p \models_\mu \Box\,\mathsf{input}(v)$ *and* $p \models_\mu \mathsf{T}\Diamond\mathsf{input}(v')$ *implies* $v = v'$, *for* $v, v' \in \{0, \frac{1}{2}, 1\}$.
(5) $p \models_\mu \neg\mathsf{echo}_2(\frac{1}{2})$.

Proof. We consider each part in turn:

(1) We consider two subcases:
- If $[\![\mathsf{input}(\frac{1}{2})]\!]_\mu(p) \in \{\mathbf{f}, \mathbf{b}\}$ then $p \models_\mu \neg\mathsf{input}(\frac{1}{2})$ and we are done.
- If $[\![\mathsf{input}(\frac{1}{2})]\!]_\mu(p) = \mathbf{t}$ then $[\![\neg\mathsf{input}(\frac{1}{2})]\!]_\mu(p) = \mathbf{f}$, contradicting the right-hand conjunct of (CaInput) (that $\neg\mathsf{input}(\frac{1}{2})$). So this subcase is impossible.

(2) We consider three subcases:
- *Suppose* $[\![\mathsf{input}(v)]\!]_\mu(p) = \mathbf{b}$.
  Then by (CaCorrect′) (for input) $[\![\mathsf{input}(v')]\!]_\mu(p) = \mathbf{b}$ for *every* $v' \in \{0, \frac{1}{2}, 1\}$ and in particular $[\![\mathsf{input}(1\text{-}v)]\!]_\mu(p) = \mathbf{b} = \neg\mathbf{b}$ so we are done.
- *Suppose* $[\![\mathsf{input}(v)]\!]_\mu(p) = \mathbf{t}$.
  Then by (CaCorrect′) (for input) $[\![\mathsf{input}(v')]\!]_\mu(p) \in \{\mathbf{t}, \mathbf{f}\}$ for *every* $v' \in \{0, \frac{1}{2}, 1\}$ and in particular $[\![\mathsf{input}(1\text{-}v)]\!]_\mu(p) \in \{\mathbf{t}, \mathbf{f}\}$. By (CaInput) and routine arguments on the truth-tables from Figure 1, $\mathsf{input}(v')$ is true for precisely *one* value $v' \in \{0, 1\}$. The result follows.
- The case that $[\![\mathsf{input}(v)]\!]_\mu(p) = \mathbf{f}$ is precisely symmetric to the case of $\mathbf{t}$.

(3) By routine logical reasoning from parts 1 and 2 of this result.

(4) Suppose $\models_\mu \Box\,\mathsf{input}(v)$ and $p \in \mathsf{Pnt}$ and $p \models_\mu \mathsf{T}\,\mathsf{input}(v')$. Then $p \models_\mu \mathsf{input}(v)$ and $p \models_\mu \mathsf{T}\,\mathsf{input}(v')$. By (CaCorrect′) (for input) and Proposition 2.2.3(8), $p \models_\mu \mathsf{T}\,\mathsf{input}(v)$. We use part 3 of this result.

(5) If $[\![\mathsf{echo}_2(\frac{1}{2})]\!]_\mu(p) \in \{\mathbf{f}, \mathbf{b}\}$ then $p \models_\mu \neg\mathsf{echo}_2(\frac{1}{2})$ and we are done. So suppose $[\![\mathsf{echo}_2(\frac{1}{2})]\!]_\mu(p) = \mathbf{t}$, i.e. $p \models_\mu \mathsf{T}\,\mathsf{echo}_2(\frac{1}{2})$; we will derive a contradiction.
  By weak modus ponens (Prop. 2.2.3(4)) and (CaEcho2?) $\models_\mu \boxdot\mathsf{echo}_1(\frac{1}{2})$. By (CaCorrect) $\models_\mu \boxdot\mathsf{TF}[\mathsf{echo}_1]$ so using Theorem 2.4.5 and Lemma 2.3.7(2) (or just direct from the 3-twined property in Definition 2.4.1) $\models_\mu \Diamond\mathsf{T}\,\mathsf{echo}_1(\frac{1}{2})$. By strong modus ponens (Prop. 2.2.3(5)) and (CaEcho1?) $\models_\mu \Diamond\mathsf{T}\,\mathsf{input}(\frac{1}{2})$. But this contradicts part 1 of this result. □

Proposition 5.3.6 (Validity) *Suppose* $\models_\mu$ ThyCA *and* $v \in \{0, \frac{1}{2}, 1\}$. *Then:*

(1) $\models_\mu \Diamond\mathsf{output}(v) \rightharpoonup (\Diamond\,\mathsf{input}(v) \vee v{=}\frac{1}{2})$.
  "If a non-faulty party outputs $v \neq \frac{1}{2}$, *then* v *was the input of some non-faulty party."*
(2) $\models_\mu \Diamond\mathsf{output}(\frac{1}{2}) \rightharpoonup (\Diamond\,\mathsf{input}(0) \wedge \Diamond\,\mathsf{input}(1))$ *(we will need this to prove part 3).*
(3) $\models_\mu \Box\,\mathsf{input}(v)$ *and* $\models_\mu \mathsf{T}\Diamond\mathsf{output}(v')$ *implies* $v{=}v'$, *for every* $v' \in \{0, \frac{1}{2}, 1\}$.
  "If all non-faulty parties have the same input, then this is the only possible output of any non-faulty party.*"*

Proof. We consider each part in turn:

(1) By strong modus ponens (Prop. 2.2.3(5)) it suffices to show that $\models_\mu \mathsf{T}\Diamond\mathsf{output}(v)$ and $v \in \{0,1\}$ implies $\models_\mu \mathsf{T}\Diamond\mathsf{input}(v)$.
  So suppose $\models_\mu \mathsf{T}\Diamond\mathsf{output}(v)$ and $v \in \{0,1\}$. By (CaOutput?) (since $v = 0$ or $v = 1$) $\models_\mu \boxdot\mathsf{echo}_2(v)$ and by (CaCorrect) $\models_\mu \boxdot\mathsf{TF}[\mathsf{echo}_2]$. Using Theorem 2.4.5 and Lemma 2.3.7(3) (or just direct from the 3-twined property in Definition 2.4.1) $\models_\mu \mathsf{T}\Diamond\mathsf{echo}_2(v)$.
  By similar reasoning using (CaEcho2?) in place of (CaOutput?), we conclude that $\models_\mu \mathsf{T}\Diamond\mathsf{echo}_1(v)$. By (CaEcho1?) $\models_\mu \mathsf{T}\Diamond\mathsf{input}(v)$ as required.

(2) By strong modus ponens (Prop. 2.2.3(5)) it suffices to show that $\models_\mu \mathsf{T}\Diamond\mathsf{output}(\frac{1}{2})$ implies $\models_\mu \mathsf{T}\Diamond\mathsf{input}(0)$ and $\models_\mu \mathsf{T}\Diamond\mathsf{input}(1)$.
  So suppose $\models_\mu \mathsf{T}\Diamond\mathsf{output}(\frac{1}{2})$. By (CaOutput′?) $\models_\mu \boxdot\mathsf{echo}_1(0) \wedge \boxdot\mathsf{echo}_1(1)$.

[35] This reflects line 1 in Figure 12.

We continue to reason as for part 1 of this result to conclude $\models_\mu \mathsf{T}(\Diamond\mathsf{input}(0) \wedge \Diamond\mathsf{input}(1))$ as required.

(3) Suppose $\models_\mu \Box\mathsf{input}(v)$ and suppose $\models_\mu \mathsf{T}\Diamond\mathsf{output}(v')$; we will show $\models_\mu \mathsf{T}(v{=}v')$, i.e. $v = v'$. There are now two sub-cases:
- *Suppose* $v' \in \{0,1\}$. By part 1 of this result $\models_\mu \mathsf{T}\Diamond\mathsf{input}(v')$. But we assumed $\models_\mu \Box\mathsf{input}(v)$, so by Lemma 5.3.5(4) $v = v'$ as required.
- *Suppose* $v' = \frac{1}{2}$. By part 2 of this result $\models_\mu \mathsf{T}\Diamond\mathsf{input}(0)$ and $\models_\mu \mathsf{T}\Diamond\mathsf{input}(1)$. But we assumed $\models_\mu \Box\mathsf{input}(v)$, so by Lemma 5.3.5(4) $0 = v = 1$, a contradiction. So this case is impossible.

(A word on notation: (CaValid1) *in Figure 14 is written "*$\models_\mu (\mathsf{TB}\Box\mathsf{input}(v) \wedge \mathsf{output}(v')) \rightharpoonup v{=}v'$*". It is a fact that this is equivalent to "*$\models_\mu \Box\mathsf{input}(v)$ *and* $\models_\mu \mathsf{T}\Diamond\mathsf{output}(v')$ *implies* $v{=}v'$*", so the statement of this part of the Lemma is accurate; we have just rephrased it for convenience.)* □

Remark 5.3.7 We take a minute to discuss the precise form of the validity properties in Proposition 5.3.6. Consider this English sentence:

"If a non-faulty party outputs $v \neq \frac{1}{2}$*, then* v *was the input of some non-faulty party."*

We render this in (CaValid2) as

$$\textsc{ThyCA} \models_\mu \Diamond\mathsf{output}(v) \rightharpoonup (\Diamond\mathsf{input}(v) \vee v{=}\tfrac{1}{2}).$$

Using strong modus ponens (Prop. 2.2.3(5)), this means that if $v \neq \frac{1}{2}$ and $[\![\mathsf{output}(v)]\!]_\mu(p) = \mathbf{t}$ for some $p \in \mathsf{Pnt}$ then $[\![\mathsf{input}(v)]\!]_\mu(p') = \mathbf{t}$ for some $p' \in \mathsf{Pnt}$. We are using $\mathbf{t}$ and $\mathbf{f}$ as the truth-values of correct (non-faulty) participants, and $\mathbf{b}$ as the truth-value for incorrect (faulty) participants, so this corresponds precisely to the English sentence quoted above.

It might be instructive to consider a subtly incorrect rendering of (CaValid1)

"If all non-faulty parties have the same input, then this is the only possible output of any non-faulty party*"*

as

$$\textsc{ThyCA} \models_\mu \boxdot\mathsf{input}(v) \rightharpoonup \mathsf{output}(v).$$

This is wrong, for two reasons:

(1) output is just a predicate-symbol; it is not a function-symbol. As such, $\mathsf{output}(v)$ does not *a priori* have to return $\mathbf{t}$ or $\mathbf{b}$ on just one value v.[36] $\mathsf{output}(v)$ could be $\mathbf{t}$ and also $\mathsf{output}(v')$ might be $\mathbf{t}$ for some other v'.

(2) More subtly, just because $\models_\mu \mathsf{T}\boxdot\mathsf{input}(v)$ holds does not *a priori* mean that $\models_\mu \mathsf{T}\Diamond\mathsf{input}(1\text{-}v)$ cannot hold. Remember that we are working over an abstract 3-twined semitopology; there might be a quorum of correct participants who $\mathbf{t}$-do $\mathsf{input}(v)$ and *also* some other correct participant who $\mathbf{t}$-does $\mathsf{input}(v')$ for some other v'.

This is why we write $\mathsf{TB}\Box\mathsf{input}(v)$ in (CaValid1) in Figure 14: it means that every participant is either faulty or, if it is not faulty, it inputs v. The use of exclusive-or $\oplus$ in (CaInput) ensures that input is functional, so this does indeed render the idea of the English sentence quoted above.

5.3.3 Liveness

We work towards Proposition 5.3.11. We will need Lemma 5.3.8 and two corollaries of it:

Lemma 5.3.8 *Suppose* $\models_\mu \textsc{ThyCA}$ *and* $v \in \{0, \frac{1}{2}, 1\}$*. Then:*

(1) $\models_\mu \boxdot\mathsf{echo}_1(v)$ *implies* $\models_\mu \mathsf{T}\diamondsuit\mathsf{echo}_1(v)$.
(2) $\models_\mu \mathsf{T}\diamondsuit\mathsf{echo}_1(v)$ *implies* $\models_\mu \Box\mathsf{echo}_1(v)$.
(3) $\models_\mu \Box\mathsf{echo}_1(v)$ *implies* $\models_\mu \mathsf{T}\boxdot\mathsf{echo}_1(v)$.

Proof. Suppose $\models_\mu \boxdot\mathsf{echo}_1(v)$. By (CaCorrect) (for echo_1) and Theorem 2.4.5 $\models_\mu \mathsf{T}\diamondsuit\mathsf{echo}_1(v)$. By (CaEcho1!) (right-hand disjunct, for $\diamondsuit\mathsf{echo}_1$) $\models_\mu \Box\mathsf{echo}_1(v)$. By (CaCorrect) (for echo_1) and Lemma 2.3.6(2)

[36] Indeed, in the case of this particular protocol output can be $\mathbf{t}$ on two values; see Example 5.3.4.

$\models_\mu \mathsf{T}\boxdot\mathsf{echo}_1(v)$. □

Corollary 5.3.9 Suppose $\models_\mu$ ThyCA and $v \in \{0, \frac{1}{2}, 1\}$. Then $\models_\mu \mathsf{echo}_2(v) \rightharpoonup \boxdot\mathsf{echo}_1(v)$.

Proof. Suppose $p \models_\mu \mathsf{T}\,\mathsf{echo}_2(v)$. By (CaEcho2?) $\models_\mu \boxdot\mathsf{echo}_1(v)$. By Lemma 5.3.8 $\models_\mu \mathsf{T}\boxdot\mathsf{echo}_1(v)$. □

Corollary 5.3.10 Suppose $\models_\mu$ ThyCA. Then:

(1) $\models_\mu \mathsf{T}\diamondsuit(\mathsf{input}(0) \wedge \mathsf{TF}[\mathsf{echo}_1]) \vee \mathsf{T}\diamondsuit(\mathsf{input}(1) \wedge \mathsf{TF}[\mathsf{echo}_1])$.
(2) $\models_\mu \mathsf{T}\diamondsuit\mathsf{echo}_1(0) \vee \mathsf{T}\diamondsuit\mathsf{echo}_1(1)$.
(3) $\models_\mu \mathsf{T}\boxdot\mathsf{echo}_1(0) \vee \mathsf{T}\boxdot\mathsf{echo}_1(1)$.
(4) $\models_\mu \Box(\mathsf{echo}_2(0) \vee \mathsf{echo}_2(1))$.
(5) $\models_\mu \mathsf{T}\boxdot(\mathsf{echo}_2(0) \vee \mathsf{echo}_2(1))$.

Proof. We consider each part in turn:

(1) Using (CaInput) $\models_\mu \Box\big(\mathsf{input}(0) \vee \mathsf{input}(1)\big)$, and using (CaCorrect) $\models_\mu \boxdot\mathsf{TF}[\mathsf{echo}_1, \mathsf{input}]$. Thus by Lemma 2.3.6(1) $\models_\mu \boxdot\big((\mathsf{input}(0) \vee \mathsf{input}(1)) \wedge \mathsf{TF}[\mathsf{echo}_1, \mathsf{input}]\big)$, and rearranging we obtain

$$\models_\mu \boxdot((\mathsf{input}(0) \wedge \mathsf{TF}[\mathsf{echo}_1, \mathsf{input}]) \vee (\mathsf{input}(1) \wedge \mathsf{TF}[\mathsf{echo}_1, \mathsf{input}])).$$

By Corollary 2.4.6 (since we assumed in Remark 5.3.1 that $(\mathsf{Pnt}, \mathsf{Opn})$ is 3-twined)

$$\models_\mu \diamondsuit(\mathsf{input}(0) \wedge \mathsf{TF}[\mathsf{echo}_1, \mathsf{input}]) \vee \diamondsuit(\mathsf{input}(1) \wedge \mathsf{TF}[\mathsf{echo}_1, \mathsf{input}]).$$

By routine calculations from Figure 1 we conclude that

$$\models_\mu \mathsf{T}\diamondsuit(\mathsf{input}(0) \wedge \mathsf{TF}[\mathsf{echo}_1]) \vee \mathsf{T}\diamondsuit(\mathsf{input}(1) \wedge \mathsf{TF}[\mathsf{echo}_1])$$

as required.

(2) We combine part 1 of this result with (CaEcho1!) (left-hand disjunct, for input) using weak modus ponens and Proposition 2.2.3(8).

(3) From part 2 of this result using Lemma 5.3.8.

(4) By part 3 of this result and (CaEcho2!) we have $\models_\mu \Box\,\exists v.\mathsf{echo}_2(v)$. By Lemma 5.3.5(5) $\models_\mu \neg\mathsf{echo}_2(\frac{1}{2})$. The result follows using (CaCorrect′) (for echo_2).

(5) From part 4 of this result using (CaCorrect) (for echo_2) and Lemma 2.3.6(2). □

Proposition 5.3.11 (Liveness) *If* $\models_\mu$ ThyCA *then* $\models_\mu \Box\,\exists v.\mathsf{output}(v)$.
"If all non-faulty parties start the protocol then all non-faulty parties eventually output a value."

Proof. By Corollary 5.3.10(5) there exists a quorum $O \in \mathsf{Opn}_{\neq\varnothing}$ such that

$$\forall p \in O.(p \models_\mu \mathsf{T}\,(\mathsf{echo}_2(0) \vee \mathsf{echo}_2(1))).$$

Note in passing that by $(\mathsf{CaEcho2}_{01})$ and Proposition 3.1.3(1.e), these two disjuncts are mutually exclusive. There are now two cases:

(1) Suppose $\forall p \in O.(p \models_\mu \mathsf{T}\,\mathsf{echo}_2(v))$ for some $v \in \{0, 1\}$, so that $\models_\mu \mathsf{T}\boxdot\mathsf{echo}_2(v)$.
Then using (CaOutput!) $\models_\mu \Box\,\mathsf{output}(v)$.

(2) Suppose there exist $p, p' \in O$ such that $p \models_\mu \mathsf{T}\,\mathsf{echo}_2(0)$ and $p' \models_\mu \mathsf{T}\,\mathsf{echo}_2(1)$, so that $\models_\mu \mathsf{T}\diamond\mathsf{echo}_2(0) \wedge \mathsf{T}\diamond\mathsf{echo}_2(1)$.
By Corollary 5.3.9 $\models_\mu \mathsf{T}\boxdot\mathsf{echo}_1(0)$ and $\models_\mu \mathsf{T}\boxdot\mathsf{echo}_1(1)$ and by (CaOutput′!) $\models_\mu \Box\,\mathsf{output}(\frac{1}{2})$.

In either case, $\models_\mu \Box\,\exists v.\mathsf{output}(v)$, as required. □

Remark 5.3.12 We continue the theme of Remark 5.3.7 and comment on some design subtleties of the logical correctness property for Liveness:

(1) This property for Liveness would be subtly incorrect:

$$\text{ThyCA} \models_\mu \boxdot\exists v.\mathsf{output}(v).$$

It might look right, but we are working over an abstract 3-twined semitopology so it might be that there is a quorum of correct participants who output some value, and *also* some other correct participant not in that quorum, who does not.

(2) The English statement of Liveness includes a proviso 'if all non-faulty parties start the protocol', which is omitted in $\Box\exists v.\mathsf{output}(v)$ (and also in $\boxdot\exists v.\mathsf{output}(v)$). This is for two reasons: first, if a party does not start the protocol then arguably from our point of view it is faulty, so the proviso trivially holds by definition; but second, our model has no notion of time so talking about when and whether a party starts or does not start doing something is just not in the scope of our analysis.

(3) Does $\Box\exists v.\mathsf{output}(v)$ not mean that *everyone* outputs a value, including faulty parties? A faulty participant p will return $\mathbf{b}$ for $\mathsf{output}(v)$ for any v, which makes $[\![\exists v.\mathsf{output}(v)]\!]_\mu(p) = \mathbf{b}$. This is a valid truth-value.

So $\Box\exists v.\mathsf{output}(v)$ elegantly and accurately captures the idea that *'every correct/non-faulty participant outputs a value'*.

5.4 Further discussion of the axioms and proofs

Remark 5.4.1 Compare (CaEcho1?) from Figure 13 with (BrEcho?) from Figure 10. Note how similar they are: the only difference, in fact, is that (CaEcho1?) uses the strong implication $\rightharpoonup$ whereas (BrEcho?) uses the weak implication $\to$. This illustrates a pattern: treating distributed algorithms as axiom-systems abstracts them, and this reveals their structure, sometimes in non-obvious ways.

In Remark 5.2.2(10) we mentioned that (CaOutput!) does not have an explicit side-condition that $a \neq \frac{1}{2}$, because this is a Lemma. As promised, here is the statement and proof:

Lemma 5.4.2 *Suppose* $\models_\mu$ ThyCA *and* $v \in \{0, \frac{1}{2}, 1\}$. *Then:*

(1) $\models_\mu \mathsf{input}(v) \rightharpoonup (v{=}0 \vee v{=}1)$.
(2) $\models_\mu \mathsf{echo}_1(v) \rightharpoonup (v{=}0 \vee v{=}1)$.
(3) $\models_\mu \mathsf{echo}_2(v) \rightharpoonup (v{=}0 \vee v{=}1)$.

Proof. We consider each part in turn, freely using strong modus ponens (Prop. 2.2.3(5)) at each $p \in \mathsf{Pnt}$:

(1) Suppose $p \models_\mu \mathsf{T}\,\mathsf{input}(v)$. Then $v = 0 \vee v = 1$ is direct from (CaInput).

(2) Suppose $p \models_\mu \mathsf{T}\,\mathsf{echo}_1(v)$. By (CaEcho1?) $p \models_\mu \mathsf{T}\Diamond\mathsf{input}(v)$, and we use part 1 of this result.

(3) Suppose $p \models_\mu \mathsf{T}\,\mathsf{echo}_2(v)$. By (CaEcho2?) $\models_\mu \boxdot\mathsf{echo}_1(v)$, and by (CaCorrect) (for echo_1) and Theorem 2.4.5 $\models_\mu \mathsf{T}\diamondsuit\mathsf{echo}_1(v)$. By Lemma 2.3.7(2) $\models_\mu \mathsf{T}\Diamond\mathsf{echo}_1(v)$ and we use part 2 of this result. □

Remark 5.4.3 There are some subtle differences in the treatment of correctness between ThyBB and ThyCA. In ThyBB we insist that ready and echo are correct on quorums (= nonempty open sets), but we do not insist that these quorums must be equal. In contrast, in ThyCA we insist that there is a single quorum on which input, echo_1, echo_2, and output are all correct.

Why the difference? The correctness properties for ThyCA in Definition 5.1.4 use a notion of 'non-faulty participant' as a participant that is correct throughout a run of the algorithm, and this is explicitly used when Liveness talks about 'all non-faulty parties' at the start and at the end of a run of the algorithm. ThyBB mentions correct participants too, of course, but the relevant correctness properties from Remark 4.2.2 are subtly more lax in that they can be phrased just in terms of correctness *at a particular predicate*, rather than correctness throughout the run. The axioms reflect this.

It might be possible to make slightly weaker correctness assumptions of ThyCA: it would suffice to just check every mention of (CaCorrect) in the proofs, and see whether a weaker axiom would work that (for example) does not assume the same quorum for all predicates. We would then just have to carefully define what we mean by a 'faulty' participant. We leave this for future work.

Such questions reflect the fact that logic is doing its job, by making things explicit and precise. In English it is easy to write 'faulty participant' or 'non-faulty participant', but what this actually *means* can be, and often is, left implicit (or left to the reader to fill in from background culture and expertise). In the logical world, such concepts can be nailed down using axioms and given precise meaning; then design decisions can be discussed — using modal logic, English, or some natural combination of the two — within this logical framework.

6 Conclusions and related and future work

6.1 Conclusions

We have illustrated how a distributed algorithm can be represented declaratively via axioms. Slogans are:

(1) A distributed protocol = a logical theory (a set of axioms) in a modal logic.
(2) A run of the protocol = a model of that modal logic theory.
(3) A possible world [37] = a participant. [38]
(4) Correctness properties of the protocol = properties of the class of all models of the theory.

We use three-valued logic, such that the third truth-value helps us to represent byzantine behaviour; and we use semitopologies to represent quorums; and then we exploit the close connection between (semi)topology and logic to capture a declarative essence of the distributed algorithm.

The result is simple, yet powerful. It is mathematically novel – three-valued modal logic over a semitopology is new, as is (to the best of my knowledge) declarative axiomatic formal specification of distributed protocols – and it illuminates the algorithm in informative ways.

It is also practically worthwhile: a high-level formal specification in logical style is useful to inform new implementations, for modularity, for updatability, to promote diversity of implementations, for verifying implementation correctness, for provable security, for analysis and formal checking of security properties, and for more. It keeps only what is essential about the object of study, and behaves as an abstract reference implementation of the logical essence of what it *is*. Any implementation can legitimately claim to be 'an implementation of Bracha Broadcast', and will necessarily enjoy the correctness properties of such, if the models that it generates satisfy the axioms in Figure 10. [39]

The examples in this paper are simple by the standards of modern distributed algorithms. This is because this paper focuses on the underlying technique: the more complex the algorithm we apply these ideas to, the *more favourable* the comparison with its declarative specification is likely to become, because logic helps us to control complexity, elide detail, and focus on essential system properties. We will return to this point later when we discuss a non-trivial application of the declarative method (to *Heterogeneous Paxos*) in Future Work below.

But here, let us review once more the toy examples in this paper on their own terms. I would argue that the axiomatic approach provides a step up in mathematical abstraction and rigour; the modal logic approach to quorums and contraquorums is conceptually clean and free of counting or combinatorial arguments; and the use of a third byzantine truth-value means that byzantine behaviour comes 'for free', in the sense that side-conditions of the form 'for a correct participant' can be elided because they are taken care of automagically by the logic. All of this leads, even for simple examples, to compact and precise axioms.

Note that we even found an error (an unintended redundancy) in the pseudocode presentation of

[37] 'Possible world' is logician's jargon for a point in the model. Recall that our logic is modal, which means that there is a collection of *possible worlds*, and predicates take truth-values *at these worlds*.
[38] Note that a possible world is *not* a run of the protocol. In this paper possible worlds are identified with participants; in [17] possible worlds consist of a pair of a participant and a round number; in [16] possible worlds are more complex, but still feature a participant. So perhaps we can refine this slogan to say in general terms that a possible world consists of a participant, plus other relevant data as required by the protocol being axiomatised. The precise structure of this data (if any) can give interesting abstract insights into the notion of logical time (if any) of the protocol.
[39] Recall for example the evolution to taking such specifications as standard practice in the world of blockchain tokens: e.g. the ERC20 token standard eips.ethereum.org/EIPS/eip-20 or the FA2 token standard archetype-lang.org/docs/templates/fa2/. The point here is cultural (not technical): it is now generally recognised that a logical specification of a token is essential. Standard formal verification tools can then be applied [15].

Crusader Agreement, as discussed in Remark 5.2.2(3). This was not obvious from reading the pseudocode and the English discussion of the source material. However, once this material was rendered into declarative axioms and treated logically, the redundancy became evident. So even on examples specifically chosen for their simplicity, the declarative analysis uncovered something new and unexpected. One wonders what else we could find, understand, optimise, and even correct if we applied this technique to larger, newer, less well-studied algorithms.

6.2 Related work

We can think of the logic of this paper as a declarative complement to TLA+, which (while also a modal logic) is more imperative in flavour. TLA+ is an industry standard which is widely and successfully used to specify and verify implementations of distributed protocols [22,23,38]. However, TLA+ is a tool for specifying transition systems. This is therefore in the spirit of imperative programming, and this paper complements that technique with a declarative approach.

Declarative specification itself is well-studied and its benefits are well-understood: this is what makes (for example) Haskell and Lisp different from C and Bash. What this paper does is show how to bring such principles to bear on a new class of distributed algorithms, of which Bracha Broadcast is a simple but canonical representative. Perhaps axioms like those in this paper might one day be executed as logic clauses in a suitable declarative programming framework, but in the first instance their natural home is likely to be as a source of truth in a specification document, in a formal verification in a theorem-prover, or in a model-checker.

In practice, including in industrial practice, broadcast and consensus protocols are typically specified in English (as in Example 2.1.1 or Remark 4.1.1) or in pseudocode (as in Figure 12). The webpage [1] or the book [9] are good examples of the genre. If formalisation is undertaken, it proceeds in TLA+ (or possibly in LEAN or a similar system), but what gets formalised is still basically a (larger or smaller) description of a (larger or smaller) concrete transition relation.

The essential feature of the logical approach in this paper is to treat the algorithm declaratively at a higher level of abstraction, as an axiomatic theory, rather than as a transition system. It is not obvious that this should preserve the essence of the algorithm, but it does.

This naturally leads to a question: suppose we are given a declaratively specified protocol ‘X’, and some (pseudo-)code implementation ‘C’ that claims to satisfy specification X. How could we prove that indeed code C does satisfy specification X? This may not be trivial (depending on the complexity of C and X), but it would just amount to lemmas proving (formally or informally, depending on the level of assurance that we want) that C satisfies X. This is not a novel scenario: in many other fields it is standard to have a concrete implementation, and an abstract (axiomatic) specification, and we wish to prove that the former instantiates the latter. Indeed one instance of this arose in my own work on cryptocurrency token implementations (as the reader can imagine, the design of these is safety-critical) [15]; the reader is referred in particular to the methodological discussion in Remark 11 and Figure 8(b) of that paper, and in particular the benefits of this methodology for proving correctness properties once-and-for-all from the axioms, and then reusing those axiom-based proofs for multiple implementations just by updating the relevant implementation-to-axiom lemmas. While the work involved was not trivial, in technical terms it was routine, and things *like* this are done in industrial research every day.

6.3 Unrelated work

There are fields adjacent to, or superficially similar to, distributed algorithms, and many applications of logic and topology. There is a danger of being misled by a keyword like ‘topology’ to imagine that this paper is doing one thing, when it is doing another.

So it might be useful to spell out what this paper is not.

(1) *This is not a paper on algebraic topology.* We mention this because algebraic topology has been applied to the solvability of distributed-computing tasks in various computational models (e.g. the impossibility of wait-free k-set consensus using read-write registers and the Asynchronous Computability Theorem; a good survey is in [19]). Semitopology is topological in flavour, but it is not the same as topology, and this is not a paper about algebraic topology applied to the solvability of distributed-

computing tasks.

(2) *This is not a paper about large examples or difficult theorems.* The examples in this paper (voting, broadcast, agreement) are toy and/or textbook algorithms. This is a feature, not a bug! The point of this approach is to attain simplicity via abstraction in logic. So for example: if the proof of Proposition 4.2.12 looks boringly similar to the proof of Proposition 5.3.3, then this is evidence that our abstraction via axiomatic logic has worked its magic and let us turn *complex informal verbal arguments* into *routine, regular symbolic ones.* This flavour of boredom is something that the field of distributed algorithms (in my opinion) needs more of.

And this simplicity via abstraction in logic is not mathematically trivial; far from it. The use of three truth-values, semitopologies, modal logic, and the precise forms of the axioms, are novel. It was hardly obvious that this combination of tools could be usefully applied to study distributed algorithms. Yet we have seen that this is so, and once we have set the machinery up, axioms become compact and precise, and proofs become — if not completely easy — at least precise, symbolic, compact, and clear.

(3) *This is not a paper on distributed programming,* it is a paper on distributed *algorithms.* Process algebras (perhaps guided by session types), or imperative programs with (say) concurrent separation logic, are valid approaches to designing distributed programs, but so far as I am aware they are no help to design (say) an efficient blockchain consensus algorithm. Those approaches are also typically operational in nature, aiming to model and reason about the dynamic behaviour of systems; whereas the approach proposed in this paper is more static, focussing on design-level evaluation rather than runtime behaviour. While that is true, the fundamental difference of process calculi, session types, and similar systems is that they are fundamentally concerned with writing correct programs, whereas the focus in this paper (and in distributed algorithms in general) is on designing specific algorithms, like 'agreement' or 'broadcast', that are resilient to faulty behaviour. [40] This is a different part of the design space, with its own distinctive literature and norms.

6.4 Reflection on the axiomatisations

(1) *Developing an axiomatisation can be hard work, even for a 'simple' protocol* (though it gets easier with practice).

The work is constructive in the sense that it comes about because we are forced to understand the protocol more deeply. An example of this (which I did not expect) was the discovery of the accidental redundant conjunct in the Crusader Agreement protocol (Remark 5.2.2(3)). This was effectively invisible in the pseudocode description, but became clear in the declarative logical axiomatisation.

It is easy enough to read a protocol and to fool ourselves into thinking that we understand it. [41] Converting it into an axiomatisation forces us to be explicit about the nature of the thing itself.

(2) I would argue that *the pseudocode in presentations like Figure 12 is not even really code, or even pseudocode.*

It is written in code-like style, but consider that clauses are 'executed' in parallel and they follow a Horn clause like structure: Horn clauses are declarative and logical in flavour (coming from logic programming), not imperative and procedural. So I would argue that the logical content is already there, even in Figure 12, and it is just struggling to escape from a code-flavoured presentation.

(3) *How do we know when an axiomatisation is 'good'?*

One useful test is: does it make the proofs easy? As is often the case, small (or large!) differences in how axioms are presented can make big differences to how cleanly proofs go through. This is a criterion for preferring one axiomatisation over another, and it is also a way to understand the

[40] A supplementary word here: *algorithms* and *programs* live on fundamentally different levels of the development stack. Quicksort is a classic list sorting algorithm; `l.sort()` is a (line in) a program; these are not equivalent. Thinking of algorithms as 'just special cases of programs' is a category error that puts the cart before the horse; similarly *distributed algorithms* are not just a special case of *distributed programs.*

[41] Though speaking for myself, I could not even manage that. When I was first exposed to these distributed protocols — written in some combination of English and pseudocode — I did not understand them *at all.* Abstracting away the pseudocode to logical axioms was my way of making sense of the field. Axioms do not lie: they mean what they mean. If you mean something else, then you tweak the syntax or change the logic.

protocol: we can say that we have truly understood a distributed protocol when we have expressed it as an axiomatisation that is simple and gives simple proofs. Finding this axiomatisation is often not trivial, but once found, its simplicity is powerful evidence that we have arrived at something *mathematically essential* about what the protocol is trying to express.

(4) *Axiomatic proofs are very precise* (certainly compared to English or pseudocode descriptions).
A predicate has an unambiguous meaning, and logical proofs are just about manipulating axioms using the rules of logic. This can be a helpful support for precision and correctness.

(5) *How canonical is the modal logic in this paper?* Might other logics be required for other protocols?
The modal logic in this paper is canonical in the sense that it seems to be minimal (this is an intuition, not a theorem) required to express Bracha Broadcast and Crusader Agreement. This is nice, because it gives some precise formal measure of how Bracha Broadcast and Crusader Agreement are similar: same logic; same models; slightly different axioms and correctness properties.
However, the logic in this paper is certainly not sufficient for all possible protocols. The Declarative Paxos axiomatisation [17] requires a different notion of possible world (a participant and a round number) *and* a different logic, featuring distinctive 'Paxos-flavoured' modalities. Heterogeneous Bracha Broadcast requires its own highly distinctive set of models and modalities [16,18].
In practice, I expect each family of protocols to induce a notion of model and modal logic. Family resemblances between protocols will be reflected in structural and syntactic similarities (or identities) between their models and logics. I would argue that this is a feature, since it brings out mathematical structure that is not always evident from reading an English description. So in this sense, several distinct natural logics emerging from several distinct protocols or protocol-families is useful.
In contrast, if we want to build tools (as we mention in Future Work below), or if we want to study classes of protocol-families, we might prefer to construct *the* encompassing, all-singing-all-dancing parametric framework into which all (or at least many) protocol logics might fit, the better to study families of logics and the better to aid users in setting up *their* favourite protocol in a convenient engineering environment. This is a different challenge and we make a start in that direction with the *Coalition Logic* framework in [17].

6.5 Algorithmic time, logical time, and causation

We have noted (e.g. in the Abstract, or just above in Subsection 6.2) that the logical approach of this paper treats a distributed algorithm declaratively as an axiomatic theory: and therefore, it does *not* treat the algorithm as a transition system.

Unlike with approaches based on LTL or TLA+ or similar logics, there is no transition system of an abstract machine; there are just possible worlds (which in this paper are just participants), and predicates taking truth-values on these worlds. This invites a question: what notions of time and causation can exist in the declarative analysis, if any?

Here, the phrases *algorithmic time* and *logical time* may be helpful: our axiomatisations eliminate algorithmic time (there are no state transitions) but we shall see that there can still be a notion of logical time, and indeed in a certain sense logical time is the true notion of time, and algorithmic time is just a side-effect of implementation.

Let us examine the axiomatisations: the voting protocol in Figure 3; Bracha Broadcast in Figure 10; and Crusader Agreement in Figure 13. We note that the backward rules can be viewed as putting predicates in a logical time ordering. For example in Figure 10, we note that:

- (BrEcho?) can be read as 'if I echo, it is *because* somebody broadcast', and
- (BrReady?) can be read as 'if I am ready, it is *because* a quorum echoed', and
- (BrDeliver?) can be read as 'if I deliver, it is *because* a quorum was ready'.

These are intuitions, not mathematical proofs: the axioms are what they are, but we can interpret them as above. Similar patterns are observable in Figures 13 and Figure 3.

In this view, the backward rules say that broadcast causes/comes before echo, which causes/comes before ready, which causes/comes before deliver. This is logical time and logical causation. [42]

[42] Do the forward rules now also give notions of logical time and causation? Yes they do — they reflect the same structure — but arguably not quite as clearly, just because forward rules tend to be more complicated.

A declarative axiomatisation of Paxos [17] has a similar structure, except that it is richer, reflecting the additional complexity of the Paxos algorithm. Declarative Paxos requires a *round number* n, which is attached to each possible world. So there, the notion of logical time is created by an interplay between the structure of axioms ordering predicates (as above), and the explicit round number carried by each possible world. [43]

I expect that this will be typical: each family of protocols will come with its own notion of logical time, which will be expressed by a combination of explicit structure on the set of possible worlds, and implicit structure visible in the backward rules.

To sum up: the declarative approach eliminates algorithmic time (by design) but retains an abstract notion of *logical time*. I would further argue that logical time is in fact more faithful to what time *essentially is* in the algorithm. Any algorithm needs to be implemented, and thus it needs to be embedded in an implementation-dependent algorithmic time, but from the point of view of this paper, algorithmic time is a byproduct of implementation, and it is not something that is necessarily essential to what the algorithm *is*. [44]

This may be a different perspective on what time is from what the reader may be used to seeing, but I think it is an interesting one and natural in its own way, once we get used to the declarative approach.

6.6 Future work

6.6.1 Other algorithms

Natural future work is to study these ideas applied to other distributed algorithms. Ones that are foundational for many modern blockchain systems include: Paxos (for this, a draft paper exists [17]), Practical Byzantine Fault Tolerance (PBFT) [10], HotStuff [37], and Tendermint [8]; another family of protocols organises transactions into a directed acyclic graph (DAG) rather than a strict chain (for parallelism and scalability), including DAG-rider [20], Narwhal and Tusk [11], and Bullshark [35].

Many other protocols and protocol-families exist, and studying what logics and families of logics they might correspond to, is an open problem whose exploration would be well worthwhile. [45]

6.6.2 Heterogeneous Paxos

In other research, we have used declarative techniques to find errors in Heterogeneous Paxos [32,33], and to design a replacement protocol [16] whose correctness has been verified in Lean 4 [18]. Empirically, we found that the power of declarative techniques played a key role: *the relevant design-space exceeded our ability to explore using pseudocode and intuition*, and our understanding of the protocol was greatly enhanced by having access to declarative abstractions. A full account of this scaled-up application of declarative techniques so far is in [16], and this is ongoing work.

Intuitively this is because moving *forwards*, we have to account for failure or branches, whereas looking *backward* we know that something has happened and we just need to explain why. This shows up even in the simple examples of this paper, e.g. in Figure 10 we have three backward rules but *four* forward rules, and in Figure 13 we have the same number of backward as forward rules but the forward rules have slightly more complex structure (an extra disjunct in (CaEcho1!); an additional existential quantifier in (CaEcho2!)). So if all we care to do is identify logical sequencing, then backward rules may give a cleaner view.

[43] If the reader prefers, we could make a finer distinction: between *logical time* and *causation*. In this view, we could say that Bracha Broadcast and Crusader Agreement have no notion of logical time but they do have a notion of causation as discussed; in contrast, Paxos does have a notion of logical time given by the round number, and within each round it also has a notion of causation which resembles the one we see in this paper. I find it simplest to just think in terms of a single notion of logical time, referring to the logical structure of how the algorithm evolves.

[44] This point is not purely philosophical. Eliminating algorithmic time leads to smaller models (because we do elide implementational detail of what transitions happened in which order). Therefore if and when model-checking tools might become available for the declarative analysis, we can anticipate that eliding algorithmic time may yield smaller search-spaces for counterexamples.

[45] To pick one of many recent examples: at time of writing, Solana had recently proposed *Alpenglow* https://forum.solana.com/t/simd-0326-proposal-for-the-new-alpenglow-consensus-protocol/4236.

6.6.3 More than three truth-values

We used three truth-values in this paper: **f** and **t** model correct behaviour, and **b** models faulty behaviour. That sufficed for voting, Bracha Broadcast, and Crusader Agreement, but there are many kinds of faulty behaviour and in other applications we may want to distinguish them: e.g. we might wish to distinguish *crash faults* (where a participant crashes permanently or for a while) from *byzantine faults* (where a participant actively interacts with other participants not according to the protocol).

Could we use a four-valued logic for this? What about a five-valued logic? What about truth-values from the interval $[0, 1]$? What about an arbitrary lattice, or Heyting algebra?

The answer to all of these questions is: yes. It is our logic, and we can do what we want. In particular, we can use whichever domain of truth-values we feel most appropriately captures what we want to express — be it two-valued, three-valued (as has been helpful in this paper), or more. [46]

6.6.4 Mechanised tools

Other natural future work is to integrate the logics thus produced into model-checkers and theorem-provers. There is nothing necessarily mathematically complex about this (which is a feature, not a bug): a three-valued logic over a semitopology is not a mathematically complex entity. For example: the TLC model checker [38] for TLA+ provides state exploration to verify safety and liveness properties; the PlusCal translator simplifies specification by offering a high-level pseudocode-like interface; and TLA+ Toolbox [21] provides a development environment. It would be good to integrate the logic from this paper into this toolchain or one like it. If there is a catch here, it is that semitopologies involve powersets, which can lead to an exponential state space explosion when looking for counterexamples, so we might do well to choose a tool that would handle this well. On this topic semitopologies have an algebraic analogue of *semiframes* [14,12]; these provide additional abstraction and so might be useful in the search for counterexamples, by somewhat reducing state space.

6.6.5 Implementation

We have seen in this paper how distributed protocols can be represented as axiomatic theories. Quorums are modelled via semitopologies; byzantine behaviour is modelled via three-valued logic; axioms are written in a simple but expressive modal logic.

By design, this elides implementational details, so the question arises: can we put these details back in? Can we go from abstract specification to concrete implementation? And if we can, could we go to a *fast* implementation?

I see these questions as being essentially about, or analogous to, questions of compilation. That is: given any declarative program or specification, can we compile it to transitions of a concrete machine — embedded in 'algorithmic time', as per Subsection 6.5 — and if so, can we make that machine run quickly?

I am extremely hopeful that this should be possible, for two reasons. First, we can already do similar things with programming. For instance, 'efficient compilers for functional programming languages' is not an oxymoron; on the contrary it is a lucrative profession. And second, when we look at the axioms of ThyBB and ThyCA we see that they are highly structured and almost like clauses in some kind of logic programming language or rewrite system. This suggests that while *for arbitrary theories* (i.e. random sets of axioms) execution may not be efficient or even possible, protocol designers are not generating protocols corresponding to arbitrary theories. On the contrary, they are generating structured theories that may consist of predicates satisfying some kind of syntactic form not dissimilar to Horn clauses, or perhaps ones which can be seen as confluent rewriting theories. There is every prospect that these may be obligingly

[46] We do need to be careful about a possible explosion in the number of possible connectives.
Two-valued logic has a space of unary connectives of size $2^2 = 4$. The space of binary connectives has size $2^{(2^2)} = 16$. This is manageable, but the number grows: n-valued logic has a space of unary connectives of size n^n, and a space of binary connectives of size $n^{(n^2)}$. If $n = 3$ (as is the case of this paper) then there are $3^9 \approx 2 * 10^4$ binary connectives. If $n = 4$ then we have $4^{16} = 2^{32} \approx 4 * 10^9$. If $n = 5$ then we have $5^{25} \approx 3 * 10^{17}$.
So we should not add truth-values just for the sake of it. However, we should also not hesitate to use a particular domain of truth-values (even a large one) if we have *specific* need of it. This is because *if* we have a particular application in mind, which requires us to use a particular domain of truth-values, then that application should also pick out the relevant modalities and connectives for us, and we are free to ignore the (possibly large) space of other connectives.

susceptible to automated generation of efficient executables. Finding out what this syntactic form may be, and constructing appropriate compilers, would be interesting future work.

References

[1] Ittai Abraham, Naama Ben-David and Sravya Yandamuri, *Asynchronous agreement part 4: Crusader agreement and binding crusader agreement*, https://decentralizedthoughts.github.io/2022-04-05-aa-part-four-CA-and-BCA/ (permalink) (2022). Online article in Decentralized Thoughts.

[2] Ittai Abraham, Naama Ben-David and Sravya Yandamuri, *Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement*, in: *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing*, PODC'22, page 381–391, Association for Computing Machinery, New York, NY, USA (2022), ISBN 9781450392624. Available online at https://eprint.iacr.org/2022/711.pdf. https://doi.org/10.1145/3519270.3538426

[3] Orestis Alpos, Christian Cachin, Björn Tackmann and Luca Zanolini, *Asymmetric distributed trust*, Distributed Computing **37**, pages 247–277 (2024). https://doi.org/10.1007/S00446-024-00469-1

[4] Ignacio Amores-Sesar, Christian Cachin and Jovana Micic, *Security analysis of Ripple consensus*, in: Q. Bramas, R. Oshman and P. Romano, editors, *24th International Conference on Principles of Distributed Systems, OPODIS 2020, December 14-16, 2020, Strasbourg, France (Virtual Conference)*, volume 184 of *LIPIcs*, pages 10:1–10:16, Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2020). https://doi.org/10.4230/LIPICS.OPODIS.2020.10

[5] Ignacio Amores-Sesar, Christian Cachin and Enrico Tedeschi, *When is spring coming? A security analysis of Avalanche consensus*, in: E. Hillel, R. Palmieri and E. Rivière, editors, *26th International Conference on Principles of Distributed Systems, OPODIS 2022, December 13-15, 2022, Brussels, Belgium*, volume 253 of *LIPIcs*, pages 10:1–10:22, Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022). https://doi.org/10.4230/LIPICS.OPODIS.2022.10

[6] Ofer Arieli, Arnon Avron and Anna Zamansky, *Ideal paraconsistent logics*, Studia Logica **99**, pages 31–60 (2011). https://doi.org/10.1007/s11225-011-9346-y

[7] Gabriel Bracha, *Asynchronous byzantine agreement protocols*, Information and Computation **75**, pages 130–143 (1987), ISSN 0890-5401. https://doi.org/10.1016/0890-5401(87)90054-X

[8] Ethan Buchman, Jae Kwon and Zarko Milosevic, *The latest gossip on BFT consensus*, CoRR (2018). https://doi.org/10.48550/arXiv.1807.04938

[9] Christian Cachin, Rachid Guerraoui and Luís E. T. Rodrigues, *Introduction to Reliable and Secure Distributed Programming (2. ed.)*, Springer (2011), ISBN 978-3-642-15259-7. https://doi.org/10.1007/978-3-642-15260-3

[10] Miguel Castro and Barbara Liskov, *Practical Byzantine Fault Tolerance*, in: M. I. Seltzer and P. J. Leach, editors, *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI), New Orleans, Louisiana, USA, February 22-25, 1999*, pages 173–186, USENIX Association (1999). https://dl.acm.org/citation.cfm?id=296824

[11] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino and Alexander Spiegelman, *Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus*, in: Y. Bromberg, A. Kermarrec and C. Kozyrakis, editors, *EuroSys '22: Seventeenth European Conference on Computer Systems, Rennes, France, April 5 - 8, 2022*, pages 34–50, ACM (2022). https://doi.org/10.1145/3492321.3519594

[12] Murdoch J. Gabbay, *Semitopology: decentralised collaborative action via topology, algebra, and logic*, College Publications (2024). ISBN 9781848904651.

[13] Murdoch J. Gabbay, *Semitopology: a topological approach to decentralised collaborative action*, The Journal of Logic and Computation (2025). https://doi.org/10.1093/logcom/exae050

[14] Murdoch J. Gabbay, *Semiframes: the algebra of semitopologies and actionable coalitions*, Mathematical Structures in Computer Science **36**, pages 1–85 (2026). https://doi.org/10.1017/S096012952510042X

[15] Murdoch J. Gabbay, Arvid Jakobsson and Kristina Sojakova, *Money Grows on (Proof-)Trees: The Formal FA1.2 Ledger Standard*, in: B. Bernardo and D. Marmsoler, editors, *3rd International Workshop on Formal Methods for Blockchains (FMBC 2021)*, volume 95 of *Open Access Series in Informatics (OASIcs)*, pages 2:1–2:14, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021), ISBN 978-3-95977-209-9, ISSN 2190-6807. https://doi.org/10.4230/OASIcs.FMBC.2021.2

[16] Murdoch J Gabbay and Isaac Sheff, *Heterogeneous trust in reliable broadcast via modal logic and history structures* (2025). https://doi.org/10.5281/zenodo.17636313

[17] Murdoch J. Gabbay and Luca Zanolini, *A declarative approach to specifying distributed algorithms using three-valued modal logic* (2025). Journal draft submitted for publication, 2502.00892. https://doi.org/10.48550/arXiv.2502.00892

[18] Anthony Hart, *Heterogeneous broadcast in Lean 4* (2025). Lean 4 formalisation of the paper *"Heterogeneous trust in reliable broadcast via modal logic and history structures"* by Gabbay and Sheff. https://doi.org/10.5281/zenodo.17611735

[19] Maurice Herlihy, Dmitry Kozlov and Sergio Rajsbaum, *Distributed computing through combinatorial topology*, Morgan Kaufmann (2013). ISBN 978-0-12-404578-1. https://doi.org/10.1016/C2011-0-07032-1

[20] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor and Alexander Spiegelman, *All you need is DAG*, in: A. Miller, K. Censor-Hillel and J. H. Korhonen, editors, *PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021*, pages 165–175, ACM (2021). https://doi.org/10.1145/3465084.3467905

[21] Markus Alexander Kuppe, Leslie Lamport and Daniel Ricketts, *The TLA+ toolbox*, in: R. Monahan, V. Prevosto and J. Proença, editors, *Proceedings Fifth Workshop on Formal Integrated Development Environment, F-IDE@FM 2019, Porto, Portugal, 7th October 2019*, volume 310 of *EPTCS*, pages 50–62 (2019). https://doi.org/10.4204/EPTCS.310.6

[22] Leslie Lamport, *The temporal logic of actions*, ACM Transactions on Programming Languages and Systems (TOPLAS) **16**, pages 872–923 (1994). https://doi.org/10.1145/177492.177726

[23] Leslie Lamport, *Specifying Systems, The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley (2002), ISBN 0-3211-4306-X. http://research.microsoft.com/users/lamport/tla/book.html

[24] Dahlia Malkhi and Michael K. Reiter, *Byzantine quorum systems*, Distributed Computing **11**, pages 203–213 (1998). https://doi.org/10.1007/S004460050050

[25] Joachim Neu, Ertem Nusret Tas and David Tse, *Ebb-and-flow protocols: A resolution of the availability-finality dilemma*, in: *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*, pages 446–465, IEEE (2021). https://doi.org/10.1109/SP40001.2021.00045

[26] Rafael Pass and Elaine Shi, *The sleepy model of consensus*, in: T. Takagi and T. Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 380–409, Springer (2017). https://doi.org/10.1007/978-3-319-70697-9_14

[27] Graham Priest, *Paraconsistent logic*, in: D. M. Gabbay and F. Guenthner, editors, *Handbook of Philosophical Logic, 2nd Edition*, volume 6, pages 287–393, Kluwer (2002). https://doi.org/10.1007/978-94-017-0460-1_4

[28] Jan Mas Rovira, *Modal logic for declarative distributed algorithms in Lean 4* (2026). Lean 4 formalisation of the paper *"Declarative distributed broadcast using three-valued modal logic and semitopologies"* by Gabbay. https://doi.org/10.5281/zenodo.18982928

[29] Nicola Santoro and Peter Widmayer, *Time is not a healer*, in: B. Monien and R. Cori, editors, *STACS 89*, pages 304–313, Springer Berlin Heidelberg, Berlin, Heidelberg (1989), ISBN 978-3-540-46098-5. https://doi.org/10.1007/BFb0028994

[30] Caspar Schwarz-Schilling, Joachim Neu, Barnabé Monnot, Aditya Asgaonkar, Ertem Nusret Tas and David Tse, *Three attacks on proof-of-stake ethereum*, in: I. Eyal and J. A. Garay, editors, *Financial*

Cryptography and Data Security - 26th International Conference, FC 2022, Grenada, May 2-6, 2022, Revised Selected Papers, volume 13411 of *Lecture Notes in Computer Science*, pages 560–576, Springer (2022). https://doi.org/10.1007/978-3-031-18283-9_28

[31] Michael Senn and Christian Cachin, *Asymmetric grid quorum systems for heterogeneous processes* (2025). https://doi.org/10.48550/arXiv.2509.12942

[32] Isaac Sheff, Xinwen Wang, Robbert van Renesse and Andrew C. Myers, *Heterogeneous Paxos: Technical report* (2020). 2011.08253. https://doi.org/10.48550/arXiv.2011.08253

[33] Isaac Sheff, Xinwen Wang, Robbert van Renesse and Andrew C. Myers, *Heterogeneous Paxos*, in: Q. Bramas, R. Oshman and P. Romano, editors, *24th International Conference on Principles of Distributed Systems (OPODIS 2020)*, volume 184 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:17, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021), ISBN 978-3-95977-176-4, ISSN 1868-8969. https://doi.org/10.4230/LIPIcs.OPODIS.2020.5

[34] Victor Shoup, *Proof of history: What is it good for?* (2022). https://www.shoup.net/papers/poh.pdf (permalink).

[35] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino and Lefteris Kokoris-Kogias, *Bullshark: DAG BFT protocols made practical*, in: H. Yin, A. Stavrou, C. Cremers and E. Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 2705–2718, ACM (2022). https://doi.org/10.1145/3548606.3559361

[36] A. Yakovenko, *Solana: A new architecture for a high performance blockchain v0.8.13*, Whitepaper (2018). https://solana.com/solana-whitepaper.pdf.

[37] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan-Gueta and Ittai Abraham, *HotStuff: BFT consensus with linearity and responsiveness*, in: P. Robinson and F. Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 347–356, ACM (2019). https://doi.org/10.1145/3293611.3331591

[38] Yuan Yu, Panagiotis Manolios and Leslie Lamport, *Model checking TLA$^+$ specifications*, in: L. Pierre and T. Kropf, editors, *Correct Hardware Design and Verification Methods, 10th IFIP WG 10.5 Advanced Research Working Conference, CHARME '99, Bad Herrenalb, Germany, September 27-29, 1999, Proceedings*, volume 1703 of *Lecture Notes in Computer Science*, pages 54–66, Springer (1999). https://doi.org/10.1007/3-540-48153-2_6

Acknowledgements

Thanks to two anonymous referees for their attention to this material. Thanks to Luca Zanolini for long conversations on distributed systems. I am particularly grateful to Jan Mas Rovira, for choosing to formalise this paper in Lean [28], for noting various typos and errors in the original material, and for many questions and observations which led to improvements in the exposition. I am also grateful to the organisers of and participants in Dagstuhl Seminar 26071 on *Behavioural Types for Resilience*, for giving me the opportunity to present some ideas from this material at the seminar *and* for detailed discussions and questions which again led to improvements in the exposition.